

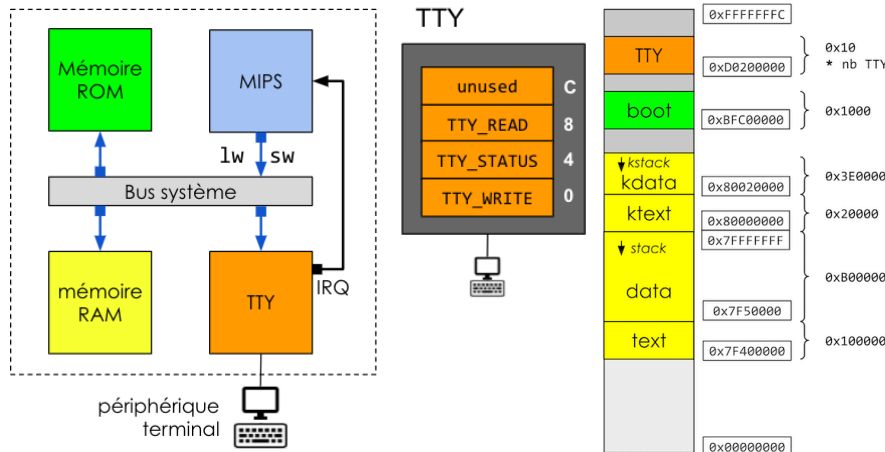
Boot et premier programme en mode kernel

1. Analyse de l'architecture

1. Analyse de l'architecture
2. Programmation assembleur
3. Compilation et édition de liens
3. Programmation en C
4. Usage de Make (optionnel)

Les trois figures ci-dessous donnent des informations sur l'architecture du prototype **almo1** sur lequel vous allez travailler.

- À gauche, vous avez un schéma simplifié.
- Au centre, vous avez la représentation des 4 registres internes du contrôleur de terminal **TTY** nécessaires pour commander un couple écran-clavier.
- À droite, vous avez la représentation de l'espace d'adressage du prototype.



Questions

1. Il y a deux mémoires dans **almo1** : RAM et ROM. Qu'est-ce qui les distinguent et que contiennent-elles ? (C9 S6+S9)
2. Qu'est-ce l'espace d'adressage du MIPS ? Quelle taille fait-il ?
Quelles sont les instructions du MIPS permettant d'utiliser ces adresses ? Est-ce synonyme de mémoire ? (C9 S7)
3. Dans quel composant matériel se trouve le code de démarrage et à quelle adresse est-il placé dans l'espace d'adressage et pourquoi à cette adresse ? (C9 S6+S7)
4. Quel composant permet de faire des entrées-sorties dans **almo1** ?
Citez d'autres composants qui pourraient être présents dans un autre SoC ? (C9 S6+connaissances personnelles)
5. Il y a 4 registres de commande dans le contrôleur de **TTY**, à quelles adresses sont-ils placés dans l'espace d'adressage ?
Comme ce sont des registres, est-ce que le MIPS peut les utiliser comme opérandes pour ses instructions (comme add, or, etc.) ?
Dans quel registre faut-il écrire pour envoyer un caractère sur l'écran du terminal (implicitement à la position du curseur) ?
Que contiennent les registres **TTY_STATUS** et **TTY_READ** ?
Quelle est l'adresse de **TTY_WRITE** dans l'espace d'adressage ? (C9 S10)
6. Le contrôleur de **TTY** peut contrôler de 1 à 4 terminaux. Chaque terminal dispose d'un ensemble de 4 registres (on appelle ça une carte de registres, ou en anglais une *register map*). Ces ensembles de 4 registres sont placés à des adresses contiguës. S'il y a 2 terminaux (**TTY0** et **TTY1**), à quelle adresse est le registre **TTY_READ** de **TTY1** ? (C9 S10)
7. Que représentent les flèches bleues sur le schéma ci-dessus ? Pourquoi ne vont-elles que dans une seule direction ? (C9 S11)

2. Programmation assembleur

L'usage du code assembleur est réduit au minimum. Il est utilisé uniquement où c'est indispensable. C'est le cas du code de démarrage. Ce code ne peut pas être écrit en C pour au moins la raison suivante : le compilateur C suppose la présence d'une pile et d'un registre du processeur contenant le pointeur de pile, or au démarrage le contenu des registres n'est pas significatif. Dans cette partie, nous allons nous intéresser à quelques éléments de l'assembleur qui vous permettront de comprendre le code en TP.

Questions

1. Nous savons que l'adresse du premier registre du **TTY** est **0xd0200000** est qu'à cette adresse se trouve le registre **TTY_WRITE** du **TTY0**. (C9 S10)
Écrivez le code permettant d'écrire le code ASCII **'x'** sur le terminal 0. Vous avez droit à tous les registres du MIPS puisqu'à ce stade il n'y pas de conventions sur leur utilisation.
Ce qu'il faut bien comprendre, c'est que l'adresse du registre **TTY_WRITE** est l'adresse d'une sortie du SoC, ce n'est pas une mémoire à proprement parler. Il est d'ailleurs interdit de lire à cette adresse. Pour écrire un message à l'écran, il faut écrire tous les caractères du message à cette adresse (0xd0200000).
2. Dans la question précédente, on connaissait l'adresse absolue du registre **TTY_WRITE**. On suppose désormais que l'adresse du premier registre du **TTY** se nomme **__tty_regs_map** (c'est le nom du label). Le programmeur ne connaît pas l'adresse, il ne connaît que le nom du label (on dit aussi symbole). Ainsi, pour écrire **'x'** sur le terminal 0, nous devons utiliser la macro instruction **la \$r, label**. Cette macro-instruction est remplacée lors de l'assemblage du code par les instructions **lui** et **ori**. Il existe aussi la macro instruction **li** qui demande de charger une valeur sur 32 bits dans un registre.

Pour être plus précis, les macro-instructions

```
la $r, label
li $r, 0x87654321
```

sont remplacées par

```
lui $r, label>>16 // chargement des 16 bits de poids forts de label
ori $r, $r, label & 0xFFFF // chargement des 16 bits de poids faible de label
lui $r, 0x8765 // chargement des 16 bits de poids fort de 0x8765431
ori $r, $r, 0x4321 // chargement des 16 bits de poids faible de 0x8765431
```

Réécrivez le code de la question précédente en utilisant `la` et `li` (C9 S34)

- En assembleur pour sauter à une adresse de manière inconditionnelle, on utilise les instructions `j label` et `jr $r`. Ces instructions permettent-elles d'effectuer un saut à n'importe quelle adresse ? Attention, la réponse n'est pas dans le cours, mais dans la connaissance du codage des instructions de saut (`jump` et `branch`) que vous avez vu au début du module.
- Vous avez utilisé les directives `.text` et `.data` pour définir les sections où placer les instructions et les variables globales, mais il existe la possibilité de demander la création d'une nouvelle section dans le code objet produit par le compilateur avec la directive `.section name,"flags"`
 - `name` est le nom de la nouvelle section. On met souvent un `.name` (avec un `.` au début) pour montrer que c'est une section et
 - `"flags"` informe sur le contenu : `"ax"` pour des instructions, `"ad"` pour des données (ceux que ça intéresse pourront regarder le manuel de l'assembleur [Assembleur/Directives/.section](#))

Écrivez le code assembleur créant la section `".mytext"` et suivi de l'addition des registres `$5` et `$6` dans `$4` (C9 S10)
- À quoi sert la directive `.globl label` dans un programme assembleur ? (C9 S34)
- Écrivez une séquence de code qui affiche la chaîne de caractère `"Hello"` sur `TTY0`. Vous devez déclarer la chaîne de caractère `"Hello"` dans la section `.data`, puis écrire le code dans la section `.text`. Vous pouvez utiliser tous les registres que vous voulez. Vous supposez que le label `__tty_regs_maps` est déjà défini et qu'il désigne le premier registre de commande du `TTY0` qui est `TTY_WRITE`. C'est une boucle qui écrit chaque caractère de la chaîne dans le registre `TTY_WRITE` du `TTY0`, jusqu'à trouver le `0` de fin de chaîne.
- En regardant le dessin de l'espace d'adressage du prototype `almo1` (plus haut et sur le slide 7 du cours 9), dites à quelle adresse devra être initialisé le pointeur de pile **pour le kernel**. Rappelez pourquoi c'est indispensable de le définir avant d'appeler une fonction C et écrivez le code qui fait l'initialisation, en supposant que l'adresse du pointeur de pile a pour nom `__kdata_end`.

3. Compilation et édition de liens

Pour obtenir le programme exécutable, nous allons utiliser :

- `gcc -o file.o -c file.c`
 - Appel du compilateur avec l'option `-c` qui demande à `gcc` de faire le *préprocessing* puis la compilation C pour produire le fichier objet `file.o`
- `ld -o bin.x -Tkernel.ld files.o ...`
 - Appel de l'éditeur de liens pour produire l'exécutable `bin.x` en assemblant tous les fichiers objets `.o`, en les plaçant dans l'espace d'adressage et résolvant les liens entre eux. Autrement dit, quand un fichier `f1.o` utilise une fonction `funf2()` ou une variable `varf2` définie dans un autre fichier `f2.o`, alors l'éditeur de liens place dans l'espace d'adressage les sections `.text` et `.data` des fichiers `f1.o` et `f2.o`, puis il détermine alors quelles sont les adresses de `funf2()` et `varf2` dans l'espace d'adressage et il complètent les instructions de `f1` qui utilisent ces adresses.
- `objdump -D file.o > file.o.s` ou `objdump -D bin.x > bin.x.s`
 - Appel du désassembleur qui prend les fichiers binaires (`.o` ou `.x`) pour retrouver le code produit par le compilateur pour le debug.

Questions sur l'édition de lien

Le fichier `kernel.ld` décrit l'espace d'adressage et la manière de remplir les sections dans le programme exécutable. Ce fichier est utilisé par l'éditeur de lien. C'est un `ldscript`, c'est-à-dire un `script` pour `ld`.

```
__tty_regs_map = [... question 1 ...] ;
__boot_origin = 0xbfc00000 ;
__boot_length = 0x00001000 ;
__ktext_origin = 0x80000000 ;
__ktext_length = 0x00020000 ;
[... question 1 ...]
__kdata_end = __kdata_origin + __kdata_length ;

MEMORY {
    boot_region : ORIGIN = __boot_origin, LENGTH = __boot_length
    ktext_region : ORIGIN = __ktext_origin, LENGTH = __ktext_length
    [... question 2 ...]
}

SECTIONS {
    .boot : {
        *(.boot)
    } > boot_region
    [... question 3 ...]
    .kdata : {
        *(.*data*)
    } > kdata_region
}
```

- Le fichier `kernel.ld` commence par la déclaration des variables donnant des informations sur les adresses et les tailles des régions de mémoire. Ces symboles n'ont pas de type et ils sont visibles de tous les programmes C. En regardant dans le dessin de la représentation de l'espace d'adressage, complétez les lignes de déclaration des variables pour la région `kdata_region`. Pour répondre, il faut savoir interpréter le dessin représentant l'espace d'adressage. (C9 S7+S38+S39)
- Le fichier contient ensuite la déclaration des régions (dans `MEMORY{...}`), c'est-à-dire les segments d'adresse en mémoire qui seront remplies par l'éditeur de lien avec les sections trouvées dans les fichiers objets selon un ordre décrit dans la partie `SECTIONS{}` du `ldscript`. Complétez cette partie (la zone `[... question 2 ...]`) pour ajouter les lignes correspondant à la déclaration de la région `kdata_region` ? (C9 S38+S39)
- Enfin le fichier décrit comment sont remplies les régions avec les sections. Complétez les lignes correspondant à la description du remplissage de la région `ktext_region`. Vous devez la remplir avec les sections `.text` issus de tous les fichiers.

Il faut bien comprendre que `.ktext` est une section produite par l'éditeur de liens. C'est ce que l'on appelle une section de sortie. `.text` est une section que l'éditeur de liens trouve dans un fichier objet `.o`, c'est ce que l'on appelle une section d'entrée. Comme il y a plusieurs fichiers objet, on doit dire à l'éditeur de lien de prendre toutes les sections `.text` de tous les fichiers qu'on lui donne. (C9 S38+S39)

3. Programmation en C

Vous savez déjà programmer en C, mais vous allez voir ici des syntaxes ou des cas d'usage que vous ne connaissez peut-être pas encore. Les questions qui sont posées ici n'ont pas toutes été vues en cours, mais vous connaissez peut-être les réponses, sinon ce sera l'occasion d'apprendre.

Questions

1. Quels sont les usages du mot clé `static` en C ? (c'est une directive que l'on donne au compilateur C)
2. Pourquoi déclarer des fonctions ou des variables `extern` ?
3. Comment déclarer un tableau de structures en variable globale ? La structure est nommée `test_s`, elle a deux champs `int` nommés `a` et `b`. Le tableau est nommé `tab` et a 2 cases.
4. Supposons que la structure `tty_s` et le tableau de registres de `TTY` soient définis comme suit. Écrivez la fonction C `int getchar0(void)` bloquante qui attend un caractère tapé au clavier sur le `TTY0`. Nous vous rappelons qu'il faut attendre que le registre `TTY_STATUS` soit différent de 0 avant de lire `TTY_READ`. `NTTYS` est un `#define` défini dans le Makefile de compilation avec le nombre de terminaux du SoC (en utilisant l'option `-D` de gcc). (C9 S10)

```
struct tty_s {
    int write;           // tty's output
    int status;          // tty's status something to read if not null)
    int read;            // tty's input
    int unused;          // unused
};
extern volatile struct tty_s __tty_regs_map[NTTYS];
// extern : parce que ce tableau n'est pas dans ce fichier
// volatile : parce que le contenu du tableau peut changer tout seul, gcc doit le lire à chaque fois
//           : cela implique que gcc ne peut pas faire d'optimisation avec les registres du MIPS
//           : (cf. note en fin de page)
```

5. Écrivez la fonction C `int puts0(char *s)` qui écrit tous les caractères de la chaîne `s` sur le terminal `TTY0`. La fonction doit rendre le nombre de caractères écrits. On suppose que les registres des TTYS sont définis comme dans la question précédente.

4. Usage de Make (optionnel)

Nous allons systématiquement utiliser des Makefiles pour la compilation du code, mais aussi pour lancer le simulateur du prototype **almo1**. Pour cette première séance, les Makefiles ne permettent pas de faire des recompilations partielles de fichiers. Les Makefiles sont utilisés pour agréger toutes les actions que nous voulons faire sur les fichiers, c'est-à-dire : compiler, exécuter avec ou sans trace, nettoyer le répertoire. Nous avons recopié partiellement le premier Makefile pour montrer sa forme et poser quelques questions, auxquels vous savez certainement répondre.

La syntaxe des `Makefile`s peut-être très complexe (c'est un vieux langage), ici nous ne verrons qu'une petite partie. Notez que le Makefile `voit` les variables du `shell` comme s'il les avait définies lui-même.

```
# Tools and parameters definitions
#
# -- options used by the prototype simulator
NTTY    ?= 2 #                default number of ttys

# -- tools
CC       = mipsel-unknown-elf-gcc #    cross-compiler MIPS
LD       = mipsel-unknown-elf-ld #    linker MIPS
OD       = mipsel-unknown-elf-objdump # desassembler MIPS
SX       = almo1.x #              prototype simulator (named almo1.x)

# -- All flags used for the gcc compiler
CFLAGS   = -c #                  stop after compilation, then produce .o
CFLAGS   += -Wall -Werror #       near all C warnings that becoming errors
CFLAGS   += -mips32r2 #           define of MIPS version
CFLAGS   += -std=c99 #            define of syntax version of C
CFLAGS   += -fno-common #         no use common sections for nostatic vars
CFLAGS   += -fno-builtin #        no use builtin func of gcc (ie strlen)
CFLAGS   += -fomit-frame-pointer # only use of stack pointer ($29)
CFLAGS   += -G0 #                 do not use global data pointer ($28)
CFLAGS   += -O3 #                 full optimisation mode of compiler
CFLAGS   += -I. #                 dir. where include <file.h> are
CFLAGS   += -DNTTYS=$(NTTY) #     number of ttys in the prototype

# Rules (here they are used such as simple shell scripts)
#
help:
@echo "\nUsage : make <compil|exec|clean> [NTTY=num]\n"
@echo "      compil : compiles all sources"
@echo "      exec   : executes the prototype"
@echo "      clean  : clean all compiled files\n"

compil:
$(CC) -o hcpua.o $(CFLAGS) hcpua.S
@$(OD) -D hcpua.o > hcpua.o.s
$(LD) -o kernel.x -T kernel.ld hcpua.o
@$(OD) -D kernel.x > kernel.x.s

exec: compil
$(SX) -KERNEL kernel.x -NTTYS $(NTTY)
```

clean:

```
-rm *.o* *.x* *~ *.log.* proc?_term? 2> /dev/null || true
```

4. Où est utilisé `CFLAGS` ? Que fait `-DNTTYS=$(NTTY)` et pourquoi est-ce utile ici ? (C9 annexe S8)

5. Si on exécute `make` sans cible, que se passe-t-il ? (C9 annexe S6)

Réponses non présentes dans les slides, mais utiles à savoir.

6. à quoi servent `@` et `-` au début de certaines commandes ?

7. Au début du fichier se trouve la déclaration des variables du Makefile, quelle est la différence entre `=`, `?:=` et `+=` ?

Note sur le mot clé `volatile`

- Quand le programme doit aller chercher une donnée dans la mémoire puis faire plusieurs calculs dessus, le compilateur optimise en réservant un registre du processeur pour cette variable afin de ne pas être obligé d'aller lire la mémoire à chaque fois. Mais, il y a des cas où ce comportement n'est pas souhaitable (il est même interdit). C'est le cas pour les données qui se trouvent dans les registres de contrôleur de périphériques. Ces données peuvent être changées par le périphérique sans que le processeur le sache, de sorte qu'une valeur lue par le processeur à l'instant `t` n'est plus la même (dans le registre du périphérique) à l'instant `t+1`. Le compilateur ne doit pas optimiser, il doit aller chercher la donnée en mémoire à chaque fois que le programme le demande.
 - `volatile` permet de dire à `gcc` que la variable en mémoire peut changer à tout moment, elle est volatile. Ainsi quand le programme demande de lire une variable `volatile` le compilateur doit toujours aller la lire en mémoire. Il ne doit jamais chercher à optimiser en utilisant un registre afin de réduire le nombre de lecture mémoire (load). De même, quand le programme écrit dans une variable `volatile`, cela doit toujours provoquer une écriture dans la mémoire (store).
 - Ainsi, les registres de périphériques doivent toujours être impérativement lus ou écrits à chaque fois que le programme le demande, parce que c'est justement ces lectures et ces écritures qui commandent le périphérique.

Boot et premier programme en mode kernel

1. Premier programme en assembleur dans la seule section de boot
2. Saut dans le code du noyau en assembleur
3. Saut dans la fonction kinit() du noyau en langage C
4. Accès aux registres de contrôle des terminaux TTY
- B5. Premier petit pilote pour le terminal

Pour les travaux pratiques, vous devez d'abord répondre à des questions qui ont pour but de vous faire lire le code et revoir les points du cours. Les réponses sont dans le cours ou dans les fichiers sources. Certaines questions ont déjà été traitées en TD, c'est normal. Ensuite, vous passez aux exercices pratiques.

La partie pratique est découpée en 5 étapes. Pour chaque étape, nous donnons (1) une brève description, (2) une liste des objectifs principaux de l'étape, (3) une liste des fichiers avec un bref commentaire sur chaque fichier, (4) une liste de questions simples dont les réponses sont dans le code, le cours ou le TD et enfin (5) un exercice de codage.

Vous allez compiler vos programmes pour le MIPS et les exécuter le prototype virtuel du SoC nommé **almo1** présenté en cours. Il est composé d'un MIPS, d'une mémoire et d'un contrôleur de terminal TTY. Le simulateur d'**almo1** se présente sous la forme d'un exécutable qui simule le comportement du SoC. Le MIPS exécute le programme se trouvant dans les bancs de mémoire du SoC. La simulation est complète depuis le signal reset et précise au cycle et au bit (cela signifie que si c'était une vraie machine, vous auriez exactement le même comportement de vos programmes).

IMPORTANT

Avant de faire cette séance, vous devez avoir regardé les documents suivants :

- Description des objectifs de cette séance et des suivantes : *obligatoire*
- Cours de démarrage présentant l'architecture matérielle et logicielle que vous allez manipuler *obligatoire*
- Éléments d'information sur les outils de la chaîne de compilation *recommandé*
- Document sur l'assembleur du MIPS et la convention d'appel des fonctions : *recommandé*
- Documentation sur le mode kernel du MIPS32 : *optionnel pour cette séance*
- Configuration de l'environnement des TP : *obligatoire si vous êtes sur votre machine personnelle, sinon lisez la suite*

Mise en place de l'environnement des TP

- **Configuration et test de l'environnement de travail sur les machines de la PPTI**
 - Éditez votre fichier `$HOME/.bashrc` et ajoutez au début:
`source /Infos/lmd/2024/licence/ue/LU3IN029-2024oct/k06/bin/SourceMe.sh`
 Ce script modifie quelques variables d'environnement telle que PATH qui permet de définir les répertoires dans lesquels le shell bash trouve ses exécutables (ici, les outils de la chaîne de compilation du MIPS et le simulateur **almo1**).
 - Exécutez pour cette fois le `.bashrc` (parce que vous venez juste de le modifier):
`source $HOME/.bashrc`
 Vous pouvez aussi ouvrir un nouveau terminal, celui-ci exécutera le script `.bashrc` avant d'afficher le prompt.
- **Récupération des codes pour le tp1**
 - Ouvrez un `terminal`
 - Créer un répertoire nommé `k06` à la racine de votre compte:
`mkdir ~/k06`
 Dans le texte des TP, nous supposons que c'est là que vous mettrez vos codes, vous pouvez choisir un autre emplacement, mais dans les textes, ce sera `~/k06`
 - Allez dans le répertoire `~/k06` :
`cd ~/k06`
 - Copier les codes du tp1:
`cp -rp /Infos/lmd/2024/licence/ue/LU3IN029-2024oct/k06/tp1 .`
 - Exécutez la commande: `tree -L 1 tp1`.
 Vous devriez obtenir ceci:

```
k06/tp1
tp1
|-- 0_test_almo1
|-- 1_hello_boot
|-- 2_init_asm
|-- 3_init_c
|-- 4_nttys
|-- 5_driver
-- Makefile
```

- **Pour tester que tout fonctionne**
 - Allez dans le répertoire `0_test_almo1`:
`cd tp1/0_test_almo1`
 - Exécutez la commande:
`make exec`
 Vous devez voir 2 fenêtres `xterm` apparaître avec le message `Hello Word` et une petite fenêtre avec une roue tournante
 - **Pour arrêter le simulateur, vous devez taper `CTRL-C` dans le terminal où vous avez démarré le simulateur.**
 - Si vous regarder dans le répertoire, le compilateur et le simulateur ont créés plusieurs fichiers (les exécutables MIPS et d'autres), pour faire le ménage et revenir aux seuls fichiers source, vous devez taper:
`make clean`

1. Premier programme en assembleur dans la seule section de boot

Nous commençons par un petit programme de quelques lignes en assembleur, placé entièrement dans la région mémoire du boot, qui réalise l'affichage du message "Hello World". C'est un tout petit programme, mais pour obtenir l'exécutable, vous devrez utiliser tous les outils de la chaîne de cross-compilation MIPS et pour l'exécuter vous devrez exécuter le simulateur du prototype. C'est simple, mais c'est nouveau pour beaucoup d'entre vous

Objectifs

- produire un exécutable à partir d'un code en assembleur.
- savoir comment afficher un caractère sur un terminal.
- analyse d'une trace d'exécution

Fichiers

- Allez dans le répertoire `1_hello_boot` : `cd ~/k06/tp1/1_hello_boot`
Il y a 3 fichiers.

```
1_hello_boot
├── hcpua.S      : code dépendant du cpu matériel en assembleur
├── kernel.ld    : ldscript décrivant l'espace d'adressage pour l'éditeur de lien
└── Makefile     : description des actions possibles sur le code : compilation, exécution, nettoyage,
```

Questions (certaines déjà vues en TD, mais vous devez ouvrir les fichiers pour répondre)

1. Dans quel fichier se trouve la description de l'espace d'adressage du MIPS ? Que trouve-t-on dans ce fichier (il faut l'ouvrir pour répondre) ?
2. Dans quel fichier se trouve le code de boot ? (h pour hardware, cpu = Central Processor Unit)
3. À quelle adresse démarre le MIPS ? Où peut-on le vérifier ?
4. Que produit `gcc` quand on utilise l'option `-c` ? (C9 S35)
5. Que fait l'éditeur de liens ? Comment est-il invoqué ? (C9 S37)
6. De quels fichiers a besoin l'éditeur de liens pour fonctionner ? (C9 S37)
7. Dans quelle section se trouve le code de boot pour le compilateur ? (la réponse est dans le code assembleur, on fera un autre choix plus tard)
8. Dans quelle section se trouve le message `"hello ..."` pour le compilateur ? Ce choix est particulier, mais ce message est en lecture seule, alors pourquoi pas...
9. Dans quelle section se trouve le code de boot dans le code exécutable produit par l'éditeur de liens ? (la réponse est dans le fichier `kernel.ld`)
10. Dans quelle région de la mémoire le code de boot est-il placé ? (la réponse est dans `kernel.ld`)
11. Comment connaît-on l'adresse du registre de sortie du contrôleur de terminal `TTY` ? (la réponse est dans `kernel.ld` et sur cette page)
12. Le code de boot se contente d'afficher un message, comment sait-on que le message est fini et que le programme doit s'arrêter ? (ou quel est le caractère de fin de chaîne en C ?)
13. Pourquoi terminer le programme par un `dead: j dead` ? Notez qu'on ne peut pas encore faire un `syscall exit` parce qu'il n'y a pas de gestionnaire de `syscall` et surtout parce `syscall` est une instruction appelée par une application utilisateur, et qu'on n'est pas dans une application utilisateur.
14. Quelle est la différence entre `#include "file.h"` et `#include <file.h>` ? Quelle option du compilateur C permet de spécifier les répertoires lesquels se trouvent les fichiers include ? Si vous ne savez pas interrogez Internet ! (C9 annexe S4+S10)
15. Comment définir une macro-instruction C uniquement si elle n'est pas déjà définie ? Écrivez un exemple. Si vous ne savez pas regardez l'usage de `#ifndef` (C9 annexe S10)
16. Comment être certain de ne pas inclure plusieurs fois le même fichier `.h` ? Vous devriez avoir trouvé l'explication en répondant à la question précédente. (C9 annexe S10)

Exercices

- Exécutez le programme en lançant le simulateur avec `make exec`, qu'observez-vous ?
- Exécutez le programme en lançant le simulateur avec `make debug`.
Cela exécute le programme pour une courte durée et cela produit deux fichiers `trace0.s` et `label0.s`.
`trace0.s` contient la trace des instructions assembleur exécutées par le processeur.
Ouvrez `trace.0.s` et repérez ce qui est cité ici
 - On voit la séquence chronologique des instructions exécutées
 - La première colonne nous informe que les adresses lues sont dans l'espace Kernel (ici, K)
 - La seconde colonne sont les numéros de cycles (ce n'est pas +1 à chaque instruction parce qu'il y a des caches et que cela prend du temps d'aller chercher les instructions dans la mémoire).
 - La troisième sont les adresses. Notez que la première instruction lue est bien en `0xbfc00000`
 - La quatrième le code binaire des instructions (sur 4 octets bien sûr)
 - Le reste de la ligne contient l'instruction désassemblée (décodée)
 - Lorsque les adresses ont un nom, c'est-à-dire qu'une étiquette leur a été attribuée, celle-ci est indiquée.
- `label0.s` contient la séquence des appels de fonctions de l'exécution. C'est en fait un extrait de la trace.
Ouvrez les fichiers `trace0.s` et `label0.s` et interprétez ce que vous voyez.
À quoi correspondent les messages `READ` et `WRITE` dans le fichier `trace0.s` ?
- Modifiez le code de `hcpua.S` afin d'afficher le message "Au revoir\n" après le message "Hello...".
Vous devez avoir un message en plus, et pas seulement étendre le premier.

2. Saut dans le code du noyau en assembleur

Rendez-vous dans le répertoire `tp1/2_init_asm`.

Dans ce deuxième programme, nous restons en assembleur, mais nous avons deux fichiers source :

1. le fichier contenant le code de boot ;
2. le fichier contenant le code du noyau. Ici, le code du noyau c'est juste une fonction `kinit()`.
Ce n'est pas vraiment une fonction car on n'utilise pas la pile.

Objectifs

- Savoir comment le programme de boot fait pour sauter à l'adresse de la routine `kinit`.
- Avoir un fichier `kernel.ld` un peu plus complet.

Fichiers

```
2_init_asm/
├── hcpua.S      : code dépendant du cpu matériel en assembleur
└── kernel.ld    : ldscript décrivant l'espace d'adressage pour l'éditeur de lien
```

— kinit.S	: fichier contenant le code de démarrage du noyau, ici c'est une routine kinit.
— Makefile	: description des actions possibles sur le code : compilation, exécution, nettoyage, etc.

Questions

1. Regarder dans le fichier `hcpua.S`, dans quelle section est désormais le code de boot ? (on a changé par rapport à l'exercice précédent)
2. Le code de boot ne fait que sauter à l'adresse `kinit` avec l'instruction `jr`, il n'y a pas de retour, ce n'est donc pas un `jal`. Où est défini l'étiquette `kinit` ? Comment le code de boot connaît-il cette adresse ? Pourquoi ne pas avoir utilisé `j init` et donc pourquoi passer par un registre ?
3. Dans `kernel.ld`, que signifie `*(.data*)` ? (C9 S38+S39)
4. Quelle est la valeur de `__kdata_end` ? Pourquoi mettre 2 «`_`» au début des variables du `ldscript` ? (réponse)

Exercices

- Exécutez le programme sur le simulateur. Est-ce différent de l'étape 1 ?
- Modifiez le code, comme pour l'étape 1, afin d'afficher un second message ?

3. Saut dans la fonction kinit() du noyau en langage C

Rendez-vous dans le répertoire `tp1/3_init_c`.

Dans ce troisième programme, nous faisons la même chose que pour le deuxième mais `kinit()` est désormais écrit en langage C. Cela change peu de choses, sauf une chose importante `kinit()` est une fonction et donc il faut absolument une pile d'exécution avant d'y entrer. Il faut donc initialiser `$29`.

Objectifs

- Savoir comment et où déclarer la pile d'exécution du noyau au démarrage (cela changera plus tard).
- Savoir comment afficher un caractère sur un terminal **depuis un programme C**.

Fichiers

3_init_c/	
— hcpua.S	: code dépendant du cpu matériel en assembleur
— kernel.ld	: ldscript décrivant l'espace d'adressage pour l'éditeur de lien
— kinit.c	: fichier en C contenant le code de démarrage du noyau, ici c'est la fonction kinit().
— Makefile	: description des actions possibles sur le code : compilation, exécution, nettoyage, etc.

Questions

1. Quand faut-il initialiser la pile ? Dans quel fichier est-ce ? Quelle est la valeur du pointeur initial ?
2. Dans quel fichier le mot clé `volatile` est-il utilisé ? Rappeler son rôle.

Exercices

- Exécutez le programme sur le simulateur. Est-ce différent de l'étape 1 ?
- Ouvrez les fichiers `kinit.o.s` et `kernel.x.s`, le premier fichier est le désassemblage de `kinit.o` et le second est le désassemblage de `kernel.x`. Dans ces fichiers, vous avez plusieurs sections. Les sections `.MIPS.abiflags`, `.reginfo` et `.pdr` ne nous sont pas utiles (elles servent au chargeur d'application, elles contiennent des informations sur le contenu du fichier et cela ne nous intéresse pas). Notez l'adresse de `kinit` dans les deux fichiers, sont-ce les mêmes ? Sont-elles dans les mêmes sections ? Expliquez pourquoi (c'est simple si vous avez compris le rôle de l'éditeur de liens).
- Modifiez le code de `kinit.c`, et comme pour l'étape 1, afficher un second message ?

4. Accès aux registres de contrôle des terminaux TTY

Rendez-vous dans le répertoire `tp1/4_nttys`.

Le prototype de SoC que nous utilisons pour les TP est configurable. Il est possible par exemple de choisir le nombre de terminaux texte (TTY). Par défaut, il y en a un, mais nous pouvons en avoir jusqu'à 4. Nous allons modifier le code du noyau pour s'adapter à cette variabilité. En outre, pour le moment, nous ne faisons qu'écrire sur le terminal, maintenant, nous allons aussi lire le clavier.

Objectifs

- Savoir comment compiler un programme C avec du code conditionnel.
- Savoir comment décrire en C l'ensemble des registres d'un contrôleur de périphérique et y accéder.

Fichiers

4_nttys/	
— hcpua.S	: code dépendant du cpu matériel en assembleur
— kernel.ld	: ldscript décrivant l'espace d'adressage pour l'éditeur de lien
— kinit.c	: fichier en C contenant le code de démarrage du noyau, ici c'est la fonction kinit().
— Makefile	: description des actions possibles sur le code : compilation, exécution, nettoyage, etc.

Questions

1. Dans le fichier `kinit.c`, dans la boucle `while` de la fonction `kinit()`, il est question d'un loopback, à quoi cela sert-il ? (Vous devez lire le code et les commentaires de cette fonction pour comprendre).
2. Dans le fichier `kinit.c`, on trouve `__tty_regs_map[ tty%NTTYS ].write = *s`, expliquez le modulo. Pour répondre à cette question, vous devez avoir compris comment sont rangés les registres de commande dans le contrôleur de TTY (C9 S10), et comprendre que le programmeur pourrait mettre n'importe quoi dans la variable `tty` et qu'on ne veut pas de bug !
3. Exécutez le programme sur le simulateur. Qu'observez-vous ? Est-ce que les deux fenêtres ont le même comportement vis-à-vis du clavier ?

Exercices

- Modifiez le code pour afficher un message sur le terminal `xterm1`, il y a toujours une attente sur le premier terminal.
- Modifiez le code pour que le programme affiche les touches tapées au clavier sur les deux terminaux. C'est-à-dire, ce que vous tapez sur le terminal `xterm0` s'affiche sur ce même terminal, et pareil pour `xterm1`. L'idée est de ne plus faire d'attente

bloquante sur le registre `TTY_STATUS` de chaque terminal. Pour que cela soit plus amusant, changez la casse (minuscule ↔ majuscule) sur le terminal `proc1_term1` (si vous tapez `bonjour 123`, il affiche `BONJOUR 123` et inversement).

B5. Premier petit pilote pour le terminal

Rendez-vous dans le répertoire `tp1/5_driver`.

Dans l'étape précédente, nous accédons aux registres de périphérique directement dans la fonction `kinit()`, ce n'est pas très simple. C'est pourquoi nous allons ajouter un niveau d'abstraction qui représente un *début* de pilote de périphérique (device driver). Ce pilote, même tout petit constitue une couche logicielle avec une API.

Objectifs

- Savoir comment créer un début de pilote pour le terminal `TTY`.
- Savoir comment décrire une API en C
- Savoir appeler une fonction en assembleur depuis le C

Fichiers

```
5_driver/
├── harch.c      : code dépendant de l'architecture du SoC, pour le moment c'est juste le pilote du TTY
├── harch.h      : API du code dépendant de l'architecture
├── hcpu.h       : prototype de la fonction clock()
├── hcpu.S       : code dépendant du cpu matériel en assembleur
├── kernel.ld    : ldscript décrivant l'espace d'adressage pour l'éditeur de lien
├── kinit.c      : fichier en C contenant le code de démarrage du noyau, ici c'est la fonction kinit().
└── Makefile     : description des actions possibles sur le code : compilation, exécution, nettoyage, etc.
```

Questions

1. Les fonctions du driver de TTY ont pour prototype:

```
/**
 * \brief   read any char from the tty until count is reached
 * \param   tty   tty number (between 0 and NTTY-1)
 * \param   buf   buffer where the read char are put
 * \param   count number of char read must be lower or equal to buf size
 * \return  the number of read chars
 */
int tty_read (int tty, char *buf, unsigned count);

/**
 * \brief   write count chars of the buffer to the tty
 * \param   tty   tty number (between 0 and NTTY-1)
 * \param   buf   buffer pointer
 * \return  the number of written char
 */
int tty_write (int tty, char *buf, unsigned count);
```

Elles sont dans `harch.h` et le code est dans le fichier `harch.c`. Dans `harch.h`, il n'y a pas la structure décrivant la carte des registres du `TTY`, laquelle est déclarée dans le `.c`. Pourquoi avoir fait ainsi ?

2. Le MIPS dispose d'un compteur de cycles internes. Ce compteur est dans un banc de registres accessibles uniquement quand le processeur fonctionne en mode `kernel`. Nous verrons ça au prochain cours, mais en attendant nous allons quand même exploiter ce compteur dans une fonction `unsigned clock(void)` dont le code est dans `hcpu.S`. Pourquoi avoir mis la fonction dans `hcpu.S` ? Rappel, pourquoi avoir mis `.globl clock`
3. Compilez et exécutez le code avec `make exec`. Observez. Ensuite ouvrez le fichier `kernel.x.s` et regardez où a été placée la fonction `clock()`. Est-ce un problème si `kinit()` n'est plus au début du segment `ktext` ? Posez-vous la question de qui a besoin de connaître l'adresse de `kinit()`

Exercices

- les fonctions `tty_read()` et `tty_write()` n'offrent pas un service suffisant, alors nous avons ajouté les fonctions `tty_getc()`, `tty_gets()` et `tty_puts()` qui permettent respectivement de lire un caractère sur un TTY, de lire une chaîne de caractères ou d'écrire une chaîne de caractères. Ces fonctions seront déplacées dans un autre fichier plus tard, mais pour le moment, elles sont dans le fichier `kinit.c`.
- Afin de vous *détendre un peu*, vous allez créer un petit jeu `guess`
 - `guess` tire un nombre entre '0' et '9' et vous devez le deviner en faisant des propositions.
 - `guess` vous dit si c'est trop grand ou trop petit. Ce programme ne va révolutionner votre vie de programmeur(se), mais bon, c'est probablement le premier programme que vous allez écrire et faire tourner sur une machine sans système d'exploitation.

Étapes

- Vous créez deux fichiers `guess.c` et `guess.h`.
 - `guess.c` contient le jeu il y a au moins une fonction `guess()`
 - `guess.h` contient les déclarations externes de `guess.c`
- `kinit()` doit lancer `guess()`
- `guess()`
 - vous demande de taper une touche pour démarrer le jeu.
 - effectue un tirage d'un nombre en utilisant la fonction `clock()` et ne gardant que le chiffre de poids faible (ce n'est pas aléatoire, mais c'est mieux que rien)
 - exécute en boucle jusqu'à réussite
 - demande d'un chiffre
 - comparaison avec le tirage et affichage des messages `"trop grand"`, `"trop petit"` ou `"gagné"`
- Vous devrez modifier le Makefile puisque vous avez un fichier à compiler en plus.
- Si c'est trop facile, vous pouvez complexifier en utilisant des nombres à 2 chiffres ou plus.

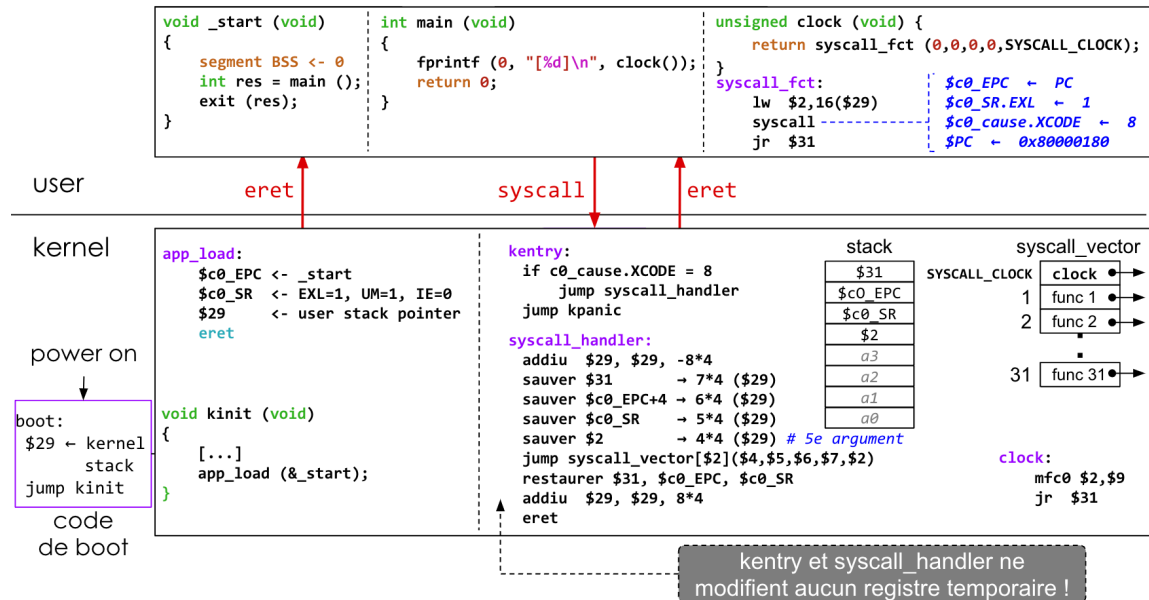
Application simple en mode utilisateur

Le schéma présenté rapidement au cours 10 (slides 26 à 31) et en détail dans l'annexe du cours 10 (slides 1 à 32) représente l'exécution d'une application utilisateur très simple dont le comportement est défini par la fonction `main()`.

L'exécution part du démarrage du SoC et va jusqu'à l'exécution de la fonction `exit()` qui stoppe l'avancée du programme.

L'objectif de ce schéma est de comprendre les interactions entre le code de boot, le noyau, l'application et les bibliothèques système. Le schéma ci-dessous ne contient pas l'intégralité du code pour des raisons évidentes de lisibilité, mais ce qui reste devrait suffire.

1. Les modes d'exécution du MIPS et les instructions système
2. Passage entre les modes kernel et user
3. Langage C pour la programmation système
4. Génération du code exécutable (optionnel)



- En bas à gauche, c'est le code de boot qui, ici, se contente d'initialiser la pile d'exécution du noyau et d'entrer dans le noyau par la fonction `kinit()` (kernel init). Ce code s'exécute en mode `kernel`, mais il ne fait pas partie du noyau car, dans un vrai système, il doit charger le noyau depuis le disque dur, mais, ici, le noyau est déjà en mémoire alors c'est plus simple.
- En bas, c'est le noyau avec la fonction `kinit()` qui initialise les structures de données internes du noyau. Ici, il s'agit juste de mettre les variables globales non initialisées à 0, puis d'appeler la routine `app_load` qui va entrer dans la première fonction de l'application utilisateur nommée `_start()`. Dans le noyau, sur la figure, on voit aussi la routine `kentry` qui est le point d'entrée du noyau pour la gestion des services. Actuellement, il n'y a que le gestionnaire d'appel système (`syscall`). Son comportement est succinctement résumé.
- En haut, c'est l'application utilisateur, décomposée en trois parties. La première à gauche est la fonction `_start()` appelée par le noyau au tout début de l'application. Cette fonction initialise à 0 les variables globales non initialisées dans le programme, puis elle appelle la fonction `main()`. Si on sort de la fonction `main()` avec un `return`, la fonction `_start` fait l'appel système `exit()`. La seconde partie au centre contient le code de l'utilisateur (notez que la fonction `main()` ou l'une des fonctions appelées par la fonction `main()` peut demander une sortie anticipée de l'application en appelant directement `exit()`). Enfin, la troisième partie, à droite, c'est le code des bibliothèques système utilisées par l'application, ce sont elles qui font les appels système, ici, seule la fonction `clock()` est représentée.

Le but de cette séance est de s'intéresser à des points particulier de ce schéma :

- D'abord, nous abordons les 2 modes d'exécution du MIPS, kernel et user, utilisés respectivement pour le noyau et l'application utilisateur.
- Puis, nous voyons les passages du noyau à l'application et de l'application au noyau.
- Ensuite, nous nous intéressons à comment écrire le code C et assembleur pour contrôler le placement en mémoire.
- Enfin, il y a quelques quelques questions comment compiler pour faciliter la compréhension des TP.

1. Les modes d'exécution du MIPS et les instructions système

Dans cette section, nous allons nous intéresser à ce que propose le processeur MIPS concernant les modes d'exécution. Ce sont des questions portant sur l'usage des modes en général et le comportement du MIPS vis-à-vis de ces modes en particulier.

Questions

1. Le MIPS propose deux modes d'exécution, rappelez quels sont ces deux modes, quel est le mode utilisé par le noyau et quel est le mode utilisé par l'application ? (C10 S6+S7)
2. Commencez par rappeler ce qu'est l'espace d'adressage du MIPS et dites ce que signifie «une adresse X est mappée dans l'espace d'adressage du MIPS». Est-ce qu'une adresse `X` mappée dans l'espace d'adressage du MIPS est toujours accessible (en lecture ou en écriture) quelque soit le mode d'exécution du MIPS. (C10 S7)
3. Le MIPS propose des registres à usage général (GPR *General Purpose Register*) pour les calculs (\$0 à \$31). Le MIPS propose un deuxième banc de registres à l'usage du système d'exploitation dans le coprocesseur 0. Chaque registre du coprocesseur 0 porte un nom correspondant à son usage, nous en avons vu 3 en cours (C10 S7+S10 à S14) : `c0_sr`, `c0_cause` et `c0_epc`. Donner leur numéro et leur rôle en une phrase ?
4. Les deux instructions qui permettent de manipuler les registres du coprocesseur 0 sont `mtc0` et `mfc0` (C10 S11). Quelle est leur syntaxe ? réponse dans Documentation MIPS Architecture et assembleur (4.) Est-ce qu'on peut manipuler ces registres de coprocesseur avec d'autres instructions ? Écrivez les instructions permettant de faire `c0_epc = c0_epc + 4` (vous utiliserez le registre GPR \$8)

- Le registre status (`c0_sr` ou `$12` du coprocesseur `0`) est composé de plusieurs champs de bits qui ont chacun une fonction spécifique.
Décrivez le contenu du registre status et le rôle des bits 0, 1 et 4 de l'octet 0. (C10 S12+S13+S15)
réponse dans Documentation MIPS Architecture et assembleur (6.)
- Le registre cause (`c0_cause` ou `$13` du coprocesseur `0`) est contient la *cause d'appel* du kernel.
Dites à quel endroit est stockée cette *cause* et donnez la signification des codes 0, 4 et 8 (C10 S14+S15)
réponse dans Documentation MIPS Architecture et assembleur (7.)
- Le registre `c0_epc` (`$14` du coprocesseur `0`) est un registre 32 bits qui contient une adresse. Vous devriez l'avoir décrit dans la question 2.
Expliquez pourquoi, dans le cas d'une exception, ce doit être l'adresse de l'instruction qui provoque une exception qui doit être stockée dans `c0_epc` ? (C10 S15)
- Nous avons vu trois instructions utilisables **seulement** lorsque le MIPS est en mode kernel, lesquelles? Que font-elles?
Est-ce que l'instruction `syscall` peut-être utilisée en mode user? (C10 S11)
- Quelle est l'adresse d'entrée dans le noyau au démarrage (à la sortie du code de boot) et après (depuis l'application) ? (C10 S15 S20)
- Que se passe-t-il lorsqu'on entre dans le noyau après de l'exécution de l'instruction `syscall` ? (C10 S15)
- Quelle instruction utilise-t-on pour sortir du noyau afin d'entrer dans l'application ?
Dites précisément ce que fait cette instruction dans le MIPS. (C10 S15)

2. Passage entre les modes kernel et user

Le noyau et l'application sont deux exécutables compilés indépendamment mais qui ne sont pas indépendants puisqu'on doit passer du noyau à l'application et inversement. Vous savez déjà que l'application appelle les services du noyau avec l'instruction `syscall`, voyons comment cela se passe vraiment depuis le code C. Certaines questions sont proches de celles déjà posées, c'est volontaire.

Questions

- Comment imposer le placement d'adresse d'une fonction ou d'une variable en mémoire lorsqu'on produit un programme binaire exécutable, c'est-à-dire quel outil de la chaîne de compilation réalise ce placement en mémoire et avec quel fichier de configuration ? (C9 S18+S22+S23 C10 annexe S6+S8)
- La première fonction d'un programme utilisateur est la fonction `_start()`, c'est elle qui appelle la fonction `main()`. La fonction `_start()` est donc dans le code de l'application, et non pas dans le noyau. Cependant le noyau doit connaître son adresse afin de pouvoir y sauter et ainsi entrer dans l'application.
Dans le code ci-après, nous voyons comment la fonction `kinit()` appelle cette fonction `_start()`. Deux fichiers sont impliqués : `kinit.c` dans lequel se trouve la fonction `void kinit(void)` et `hcpu.S` dans lequel se trouve la fonction `void app_load(void *)` en charge d'appeler la fonction `_start()`.

```
kinit.c:
void kinit (void)
{
    [...]
    extern int _start;           # declaree ailleurs a une adresse connue de l'éditeur de lien
    app_load (&_start);         # appel de la fonction app_load definie dans hcpu.S
}

hcpu.S:
.globl app_load
app_load:
    mtc0 $4, $14                # $4 contient l'argument
    li $26, 0x12                # $26 <-- 0x12 == 0b00010010
    mtc0 $26, $12               # c0_sr <-- 0x12
    la $29, __data_end          # initialisation du pointeur de pile
    eret
```

Comme le noyau et l'application sont deux exécutables compilés indépendamment, il doit y avoir une convention permettant au noyau de savoir quelle est l'adresse de `start()`.

Où se trouve donc la fonction `_start()` et comment le kernel connaît-il son adresse ? (C10 S30+S32)

À quoi sert `.globl app_load` ? (C9 S18 C10 S20)

Quels sont les registres utilisés dans le code de `app_load` ?

Que savez-vous de l'usage de `$26` ? Quels sont les registres modifiés ? Expliquez pour chacun la valeur affectée.

Que fait l'instruction `eret` ? (C10 S15)

- Que doit-on faire dans la fonction `_start()` avant l'exécution de la fonction `main()` du point de vue de l'initialisation? Et que doit-on faire dans la fonction `_start()` au retour de la fonction `main()` ? (C10 S24)
- Nous avons vu que le noyau est sollicité par des demandes de service, quels sont-ils ? Nous rappelons que l'instruction `syscall` initialise les champs `xcode` du registre `c0_cause`, ainsi donc comment le noyau fait-il pour connaître la cause de son appel? (C10 S25)
- On rappelle que `$26` et `$27` sont deux registres GPR temporaires **réservés** pour le noyau pour faire des calculs sans qu'il ait besoin de les sauvegarder dans la pile. **Ce ne sont pas des registres du coprocesseur 0** comme `c0_sr` ou `c0_epc`. En effet, l'usage de ces registres (`$26` et `$27`) par l'utilisateur ne provoque pas d'exception du MIPS. Toutefois, si le noyau est appelé alors il modifie ces registres et donc l'utilisateur perd leur valeur.
Le code assembleur ci-après contient les instructions exécutées à l'entrée dans le noyau, quelle que soit la cause. Les commentaires présents dans le code ont été volontairement retirés (ils sont dans le cours et dans les fichiers du TP). La section `.kentry` est placée à l'adresse `0x80000000` par l'éditeur de lien, conformément à ce qui est demandé dans son fichier `ldscript kernel.ld`.

kernel/hcpu.S

```
15 .section .kentry,"ax"
16 .org 0x180
22
23 kentry:
24
25 mfc0 $26, $13
26 andi $26, $26, 0x3C
27 li $27, 0x20
28 bne $26, $27, kpanic
```

Ligne 16, la directive `.org DEP` (`.org` pour *origine*, `DEP` pour *déplacement*) permet de placer le pointeur de remplissage de la section courante à `DEP` octets du début de la section, ici `DEP = 0x180`. Pourquoi faire ça ? Aurait-on pu remplacer le `.org`

0x180 par `.space 0x180` ? (C10 S5 et connaissance de l'assembleur) Expliquer les lignes 25 à 28. (C10 S20+S26+S31)

6. Le gestionnaire de `syscall` est la partie du code noyau qui gère l'exécution des services demandés par l'instruction `syscall`. Pour ce noyau, c'est un code en assembleur présent dans le fichier `kernel/hcpua.S` que nous allons détailler. Pour vous aider dans la compréhension du code, vous devez vous souvenir que l'instruction `syscall` réalise un peu un appel de fonction:
- sauf que la fonction est définie par un *numéro de syscall* contenu dans le registre GPR `$2` ;
 - les arguments sont bien dans les registres `$4` à `$7`, mais il y en a 4 au maximum;
 - toutefois, la fonction appelante de `syscall` n'a pas réservé d'espace dans la pile pour les arguments, il faudra le faire;
 - enfin, le registre `$2` contient la valeur de retour du `syscall`.

Le numéro contenu dans le registre `$2` est utilisé par le noyau pour indexer un tableau de pointeurs de fonctions de `syscall` nommé `syscall_vector[]`, ou vecteur de syscalls en français. Ce vecteur de syscalls est défini dans le fichier `kernel/ksyscalls.c`.

Les lignes 36 à 43 du code assembleur (`kernel/hcpua.S`) sont chargées d'allouer de la place dans la pile, nous allons voir pourquoi...

`common/syscalls.h`

```
1 #define SYSCALL_EXIT      0
2 #define SYSCALL_READ      1
3 #define SYSCALL_WRITE     2
4 #define SYSCALL_CLOCK     3
5 #define SYSCALL_NR       32
```

`kernel/ksyscalls.c`

```
void *syscall_vector[] = {
    [0 ... SYSCALL_NR - 1] = unknown_syscall,
    [SYSCALL_EXIT]         = exit,
    [SYSCALL_READ]         = tty_read,
    [SYSCALL_WRITE]        = tty_write,
    [SYSCALL_CLOCK]        = clock,
};
```

`kernel/hcpua.S`

```
34 syscall_handler:
35
36     addiu    $29,    $29,    -8*4
37     mfc0     $27,    $14
38     mfc0     $26,    $12
39     addiu    $27,    $27,    4
40     sw       $31,    7*4($29)
41     sw       $27,    6*4($29)
42     sw       $26,    5*4($29)
43     sw       $2,     4*4($29)
44     mtc0     $0,     $12
45
46     la       $26,    syscall_vector
47     andi     $2,     $2,    SYSCALL_NR-1
48     sll      $2,     $2,    2
49     addu     $2,     $26,    $2
50     lw       $2,     0($2)
51     jalr     $2
52
53     lw       $26,    5*4($29)
54     lw       $27,    6*4($29)
55     lw       $31,    7*4($29)
56     mtc0     $26,    $12
57     mtc0     $27,    $14
58     addiu    $29,    $29,    8*4
59     eret
```

Dessinez l'état de la pile après l'exécution de ces instructions. Que fait l'instruction ligne 44 et quelle conséquence cela a-t-il? Que font les lignes 46 à 51? Et enfin que font les lignes 53 à 59 sans détailler ligne à ligne. (C10 S26+S31+S34)

3. Langage C pour la programmation système

La programmation en C, vous connaissez, mais quand on programme pour le noyau il y a des éléments de syntaxe ou des besoins spécifiques que vous ne connaissez peut-être pas. Pour répondre aux questions, vous devez avoir lu les transparents de l'annexe du cours 10, dans lesquels une séquence complète de code est détaillée du boot à exit.

Questions

1. En assembleur, vous utilisez les sections prédéfinies `.data` et `.text` pour placer respectivement les *data* et le *code*, mais vous pouvez créer vos propres sections avec la directive `.section` (nous avons utilisé cette possibilité pour la section `.boot`). Il est aussi possible d'imposer ou de créer des sections en langage C avec la directive `__attribute__((section("section-name")))`. La directive du C `__attribute__` permet de demander certains comportements au compilateur. Ici, c'est la création d'une section, mais il y a beaucoup d'attributs possibles (si cela vous intéresse vous pouvez regarder dans la [doc de GCC sur les attributs](#)). Comment créer la section `.start` en C ? (C10 S30 C10 annexe S8)
2. En C, vous savez que les variables globales sont toujours initialisées, soit explicitement dans le programme lui-même, soit implicitement à la valeur `0`. Les variables globales initialisées sont placées dans la section `.data` (ou plutôt dans l'une des sections `data` : `.data`, `.sdata`, `.rodata`, etc.) et elles sont présentes dans le fichier objet (`.o`) produit par le compilateur. En revanche, les variables globales non explicitement initialisées ne sont pas présentes dans le fichier objet. Ces dernières sont placées dans un segment de la famille ``.bss``. Le fichier `ldscript` permet de mapper l'ensemble des segments en mémoire. Pour pouvoir initialiser à `0` les segments `bss` par programme, il nous faut connaître les adresses de début et de fin où ils sont placés en mémoire.

Le code ci-dessous est le fichier ldscript du kernel `kernel.ld` (nous avons retiré les commentaires mais ils sont dans les fichiers).

```
1 SECTIONS
2 {
3     .boot : {
4         *(.boot)
5     } > boot_region
6     .ktext : {
7         *(.text*)
8     } > ktext_region
9     .kdata : {
10        *(.*data*)
11        . = ALIGN(4);
12        __bss_origin = .;
13        *(.*bss*)
14        . = ALIGN(4);
15        __bss_end = .;
16    } > kdata_region
17 }
```

Expliquez ce que font les lignes 11, 12 et 15 ? (C10 S32)

3. Nous connaissons les adresses des registres de périphériques. Ces adresses sont déclarées dans le fichier ldscript `kernel.ld`. Ci-après, nous avons la déclaration de la variable de ldscript `__tty_regs_map`. Cette variable est aussi utilisable dans les programmes C, mais pour être utilisable par le compilateur C, il est nécessaire de lui dire quel type de variable c'est, par exemple une adresse d'entier ou une adresse de tableau d'entiers, Ou encore, une adresse de structure.

Dans le fichier `kernel.ld` :

```
__tty_regs_map = 0xd0200000 ; /* tty's registers map, described in devices.h */
```

Dans le fichier `harch.c` :

```
12 struct tty_s {
13     int write;           // tty's output address
14     int status;          // tty's status address something to read if not null)
15     int read;            // tty's input address
16     int unused;          // unused address
17 };
18
19 extern volatile struct tty_s __tty_regs_map[NTTYS];
```

Si `NTTYS` est une macro dont la valeur est `2`, quelle est l'adresse en mémoire `__tty_regs_map[1].read` ? À quoi servent les mots clés `extern` et `volatile` ? (C10 annexe S23 et connaissance du C)

4. Certaines parties du noyau sont en assembleur. Il y a au moins les toutes premières instructions du code de boot (démarrage de l'ordinateur) et l'entrée dans le noyau (kentry) après l'exécution d'un syscall. Le gestionnaire de syscall est écrit en assembleur et il a besoin d'appeler une fonction écrite en langage C. Ce que fait le gestionnaire de syscall est :
- trouver l'adresse de la fonction C qu'il doit appeler pour exécuter le service demandé;
 - placer cette adresse dans un registre, nous utilisons le registre `$2`;
 - exécuter l'instruction `jal` (ici, `jal $2`) pour appeler la fonction.

Que doivent contenir les registres `$4` à `$7` et comment doit-être la pile et le pointeur de pile? (Connaissance assembleur)

5. Vous avez appris à écrire des programmes assembleur, mais parfois il est plus simple, voire nécessaire, de mélanger le code C et le code assembleur. Dans l'exemple ci-dessous, nous voyons comment la fonction `syscall()` est écrite. Cette fonction utilise l'instruction `syscall`.

Deux exemples d'usage de la fonction `syscall()` pris dans le fichier `tp2/4_libc/ulib/libc.c`.

```
1 int fprintf (int tty, char *fmt, ...) // tty identifiant du terminal
2 {                                     // fmt chaîne format, suivie d'arguments optionnels
3     int res;
4     char buffer[PRINTF_MAX];
5     va_list ap;
6     va_start (ap, fmt);
7     res = vsnprintf(buffer, sizeof(buffer), fmt, ap); // définit le dernier argument non-optionnel // remplit le buffer avec la chaîne à
// afficher
8     res = syscall (tty, (int)buffer, 0, 0, SYSCALL_TTY_PUTS); // ← appel système
9     va_end(ap);
10    return res;
11 }
12
13 void exit (int status)
14 {
15     syscall( status, 0, 0, 0, SYSCALL_EXIT); // ← appel système
16 }
```

Le code de la fonction `syscall()` en **assembleur** est dans le fichier C : `tp2/4_libc/ulib/crt0.c`

```
1 // int syscall (int a0, int a1, int a2, int a3, int syscall_code)
2 __asm__ (
3     ".globl syscall\n"
4     "syscall:\n"
5     "    lw $2,16($29)\n"
6     "    syscall\n"
7     "    jr $31\n"
8 );
```

Combien d'arguments a la fonction `syscall()` ?

Comment la fonction `syscall()` reçoit-elle ses arguments ?

A quoi sert la ligne 3 de la fonction `syscall()` et que se passe-t-il si on la retire ?

Expliquer la ligne 5 de la fonction `syscall()`.

Aurait-il été possible de mettre le code de la fonction `syscall()` dans un fichier `.S` ? (C10 S31)

4. Génération du code exécutable (optionnel)

Pour simuler le logiciel, il faut produire deux exécutables. Nous utilisons, ici, un Makefile hiérarchique et des règles explicites. Cela sort du cadre de l'architecture, mais vous avez besoin de ce savoir-faire pour comprendre le code, alors allons-y.

Questions

1. Rappelez à quoi sert un Makefile? (C9 annexe S5 à S7)
2. Vous n'allez pas à avoir à écrire un Makefile complètement. Toutefois, si vous ajoutez des fichiers source, vous allez devoir les modifier en ajoutant des règles. Nous avons vu brièvement la syntaxe utilisée dans les Makefiles de ce TP. Les lignes qui suivent sont des extraits de `1_klibc/Makefile` (le Makefile de l'étape-1). Dans cet extrait, quelles sont la `cible` finale, les `cibles` intermédiaires et les `sources` ? A quoi servent les variables automatiques de make? Dans ces deux règles, donnez-en la valeur. (C9 annexe S5 à S7)

```
kernel.x : kernel.ld obj/hcpua.o obj/kinit.o obj/klibc.o obj/harch.o
$(LD) -o $@ -T $^
$(OD) -D $@ > $@.s

obj/hcpua.o : hcpua.S hcpu.h
$(CC) -o $@ $(CFLAGS) $<
$(OD) -D $@ > $@.s
```

3. Dans le TP, à partir de la deuxième étape, nous avons trois répertoires de sources `kernel`, `ulib` et `uapp`. Chaque répertoire contient une fichier `Makefile` différent destiné à produire une `cible` différente grâce à une règle nommée `compil`, c.-à-d. si vous tapez `make compil` dans un de ces répertoires, cela compile les sources locales.

Il y a aussi un Makefile dans le répertoire racine `4_libc`. Dans ce dernier Makefile, une des règles est destinée à la compilation de l'ensemble des sources dans les trois sous-répertoires. Cette règle appelle récursivement la commande `make` en donnant en argument le nom du sous-répertoire où descendre :

`make -C <répertoire> [cible]` est équivalent à `cd <répertoire>; make [cible] ; cd ..`

Ecrivez la règle `compil` du fichier `4_libc/Makefile`. (Ce n'est pas dit dans le cours, mais la question contient la réponse...)

```
4_libc/
├── Makefile      : Makefile racine qui invoque les Makefiles des sous-répertoires et qui exécute
├── common       : répertoire des fichiers commun kernel / user
├── kernel       : Répertoire des fichiers composant le kernel
│   └── Makefile  : description des actions possibles sur le code kernel : compilation et nettoyage
├── uapp         : Répertoire des fichiers de l'application user seule
│   └── Makefile  : description des actions possibles sur le code user : compilation et nettoyage
└── ulib        : Répertoire des fichiers des bibliothèques système liés avec l'application user
    └── Makefile  : description des actions possibles sur le code user : compilation et nettoyage
```

Application simple en mode utilisateur

Le TP est découpé en 4 étapes. Pour chaque étape, nous donnons :

1. une brève description avec une liste des objectifs principaux de l'étape,
2. une liste des fichiers avec un bref commentaire sur chaque fichier,
3. une liste de questions simples dont les réponses sont dans le code, le cours ou le TD, 4
4. un petit exercice de codage.

1. Ajout d'une bibliothèque de fonctions standards pour le kernel (klibc)
2. Programme utilisateur mais exécuté en mode kernel
3. Programme utilisateur utilisé en mode user mais sans libc
4. Ajout de la librairie C pour l'utilisateur

IMPORTANT

Avant de faire cette séance, vous devez avoir regardé les documents suivants :

- Description des objectifs de cette séance et des suivantes : *déjà lu*
- Cours sur l'exécution d'une application en mode user *obligatoire*
- Éléments d'information sur les outils de la chaîne de compilation *recommandé*
- Document sur l'assembleur du MIPS et la convention d'appel des fonctions : *recommandé, mais déjà lu*
- Documentation sur le mode kernel du MIPS32 : *obligatoire pour compléter le cours*

Récupération du code du TP

- Téléchargez **l'archive code du tp2** et placez là dans le répertoire `$HOME/k06`
- Ouvrez un `terminal`
- Allez dans le répertoire `k06` : `cd ~/k06`
- Décompressez l'archive du tp2 : `tar xvfz tp2.tgz`
- Exécutez la commande : `cd ; tree -L 1 k06/tp2`.
Vous devriez obtenir ceci :

```
k06/tp2
├── 1_klibc
├── 2_appk
├── 3_syscalls
├── 4_libc
└── Makefile
```

Objectif de la séance

Les applications de l'utilisateur s'exécutent en mode user. Dans la séance précédente, nous avons vu que les registres de commande des contrôleurs de périphériques sont placés dans l'espace d'adressage du processeur. Les adresses de ces registres ont été placées dans la partie de l'espace d'adressage interdite en mode user. Ainsi, une application n'a pas un accès direct aux périphériques, elle doit utiliser des appels système (avec l'instruction `syscall`) pour demander au noyau du système d'exploitation de faire l'accès. C'est ce que nous allons voir.

Vous allez suivre 4 étapes :

- **1_klibc**
→ Ajout d'une librairie de fonctions standards pour le noyau (klibc), mais pas d'application utilisateur ;
- **2_appk**
→ Appel d'une application utilisateur par la fonction d'initialisation `kinit()`, mais sans gestionnaire des appels systèmes dans le noyau, il n'est donc pas possible d'utiliser `syscall` ;
- **3_syscalls**
→ Ajout du gestionnaire des appels système et une application, mais **sans** la librairie de fonctions standards utilisateur (libc);
- **4_libc**
→ Ajout d'une librairie de fonctions standards utilisateur minimaliste (libc) et d'une application.

Pour les travaux pratiques, vous devez d'abord répondre aux questions, elles ont pour but de vous faire lire le code et revoir les points du cours et vous guidez, un peu, pour l'exercice. Les réponses sont dans le cours ou dans les fichiers sources. Certaines ont déjà été traitées en TD, c'est normal. Ensuite, vous passez aux exercices pratiques.

Le code se trouve normalement dans `k06/tp2/` (sinon téléchargez-le), ouvrez un terminal et allez-y. Dans ce répertoire, vous avez 4 sous-répertoires et un Makefile. Le fichier `k06/tp2/Makefile` qui fait un appel récursif à `make` et qui permet de faire le ménage en appelant les Makefiles des sous-répertoires avec la cible `clean`.

1. Ajout d'une bibliothèque de fonctions standards pour le kernel (klibc)

Objectifs de l'étape

Le noyau gère les ressources matérielles et logicielles utilisées par les applications. Il a besoin de fonctions standards pour réaliser des opérations de base, telles qu'une fonction `print` ou une fonction `rand` (générateur de nombres pseudo-aléatoires). Ces fonctions ne sont pas très originales, mais elles recèlent des subtilités que vous ne connaissez peut-être pas encore, vous pouvez les regarder par curiosité, même si ce n'est pas le but du TP. En outre, nous allons utiliser un Makefile définissant un graphe de dépendance explicite entre les fichiers cibles et les fichiers sources avec des règles de construction.

Fichiers

```
1_klibc/
├── kinit.c      : fichier contenant la fonction de démarrage du noyau
├── harch.h     : API du code dépendant de l'architecture
└── harch.c     : code dépendant de l'architecture du SoC
```

— hcpu.h	: prototype de la fonction clock()
— hcpua.S	: code dépendant du cpu matériel en assembleur
— kernel.ld	: ldscript décrivant l'espace d'adressage pour l'éditeur de lien
— klibc.h	: API de la klibc
— klibc.c	: fonctions standards utilisées par les modules du noyau
— Makefile	: description des actions possibles sur le code : compilation, exécution, nettoyage, etc.

Questions

- Ouvrez le fichier Makefile (vous pouvez regarder les dépendances en ouvrant quelques fichiers sources), puis dessiner le graphe de dépendance de `kernel.x` vis-à-vis de ses sources?
La réponse peut-être visible avec la commande `dot -Tpng Makefile.dot -oMakefile.png` à partir du fichier `Makefile.dot` reproduit ci-après (`dot` est un outil de *graphviz* ... essayez c'est magique :-)

`Makefile.dot`

```
digraph G {
  node [shape=box color=brown]
  gcc1[label="gcc -c"];
  gcc2[label="gcc -c"];
  gcc3[label="gcc -c"];
  gcc4[label="gcc -c"];
  ld[label="ld"];
  node [shape=ellipse color=blue]
  "hcpua.S" , "hcpu.h"          -> gcc1 -> "obj/hcpua.o" -> ld -> "kernel.x"
  "kinit.c" , "klibc.h" , "harch.h" , "hcpu.h" -> gcc2 -> "obj/kinit.o" -> ld
  "klibc.c" , "klibc.h" , "harch.h" , "hcpu.h" -> gcc3 -> "obj/klibc.o" -> ld
  "harch.c" , "klibc.h" , "harch.h" , "hcpu.h" -> gcc4 -> "obj/harch.o" -> ld
}
```

- Dans quel fichier se trouvent les codes dépendant du MIPS (donc écrits en assembleur) ?

Exercices

- Dans un SoC, on peut avoir plusieurs processeurs. Le noyau a besoin de savoir sur quelle instance de processeur il s'exécute. Le numéro du processeur est dans les 12 bits de poids faible du registre du **coprocesseur-0 \$15 (c0_cpuid)** (à côté des registres `c0_epc`, `c0_sr`, etc. ce n'est pas le registre GPR \$15). Ajoutez la fonction `int cpuid(void)` qui lit le registre `c0_cpuid` et qui rend un entier contenant juste les 12 bits de poids faible. Vous pouvez vous inspirer fortement de la fonction `int clock(void)`. Comme il n'y a qu'un seul processeur dans cette architecture, `cpuid` rend toujours `0`.
Ecrivez un programme de test (vous devrez modifier les fichiers `hcpu.h` pour y mettre le prototype de la fonction, `hcpua.S` pour y mettre le code et `kinit.c` pour appeler la fonction)

2. Programme utilisateur mais exécuté en mode kernel

Objectifs de l'étape

Nous allons désormais avoir deux exécutables: le noyau et l'application. Dans cette étape, nous allons voir comment le noyau fait pour appeler l'application, alors même que celle-ci n'est pas compilée en même temps que le noyau. Nous allons passer du noyau à l'application à la fin de la fonction `kinit()`.

Nous allons donc entrer dans l'application, en revanche, dans cette étape, nous n'allons pas mettre en place la gestion des syscalls. **C'est-à-dire qu'il ne sera pas possible de revenir dans le noyau depuis l'application.** C'est bien entendu une étape intermédiaire, parce qu'il faudra absolument pouvoir invoquer le noyau depuis l'application pour accéder aux périphériques. **Pour pouvoir quand même accéder aux registres de périphériques, nous allons EXCEPTIONNELLEMENT exécuter l'application en mode kernel. Ainsi, l'application pourra accéder aux adresses de l'espace d'adressage réservées au mode kernel.**

Nous avons deux exécutables à compiler et donc deux `Makefile`s de compilation. Nous avons aussi un `Makefile` qui invoque récursivement les `Makefile`s de compilation.

Fichiers

2_appk/	
— Makefile	: Makefile racine qui invoque les Makefiles des sous-répertoires et qui exécute
— kernel	— Répertoire des fichiers composant le kernel
— kinit.c	: fichier contenant la fonction de démarrage du noyau
— harch.h	: API du code dépendant de l'architecture
— harch.c	: code dépendant de l'architecture du SoC
— hcpu.h	: prototype de la fonction clock()
— hcpua.S	: code dépendant du cpu matériel en assembleur
— klibc.h	: API de la klibc
— klibc.c	: fonctions standards utilisées par les modules du noyau
— kernel.ld	: ldscript décrivant l'espace d'adressage pour l'édition de liens du kernel
— Makefile	: description des actions possibles sur le code kernel : compilation et nettoyage
— user	— Répertoire des fichiers composant l'application user
— crt0.c	: fonctions d'interface entre kernel et user, pour le moment : <code>_start()</code>
— main.c	: fonction principale de l'application
— user.ld	: ldscript décrivant l'espace d'adressage pour l'édition de liens du user
— Makefile	: description des actions possibles sur le code user : compilation et nettoyage

Questions

- Combien de fichiers de type ldscript avons-nous en tout ?
- Dans quel fichier se trouve la première fonction de l'application et comment s'appelle-t-elle?
- Quelle est la fonction du noyau qui appelle cette fonction et dans quel fichier?

4. Comment le noyau fait-il pour démarrer l'application en mode `kernel` ? (la réponse est dans la fonction de la question précédente).

Exercice

- Vous n'allez pas faire grand-chose pour cette étape parce qu'il est impossible de revenir dans le noyau après l'entrée dans l'application... tant qu'il n'y a pas le gestionnaire de syscalls dans le noyau. En conséquence, affichez juste un second message depuis la fonction `main()`.

3. Programme utilisateur utilisé en mode user mais sans libc

Objectifs de l'étape

Le programme utilisateur doit absolument s'exécuter en mode user et il doit passer par des appels système pour accéder aux services du noyau. Les services, ici, sont limités (l'accès au TTY, exit et clock), il n'empêche que pour gérer ces appels, il faut analyser la cause d'appels à l'entrée du noyau et un gestionnaire de `syscall`. Il faut aussi le gestionnaire d'exceptions, parce que s'il y a une erreur de programmation, le noyau doit afficher quelque chose pour aider le programmeur.

Le passage de l'application au noyau par le biais de l'instruction `syscall` impose que les numéros de services soient identiques pour le noyau et pour l'application. Ces numéros de service (comme `SYSCALL_TTY_PUTS`, `SYSCALL_EXIT`) sont définis dans le fichier `syscall.h` communs au noyau et à l'application. Ce fichier est mis dans un répertoire à part nommé `common`. Il n'y a qu'un seul fichier ici, mais dans un système plus élaboré, il y en a d'autres.

Fichiers

```
3_syscalls/
├── Makefile          : Makefile racine qui invoque les Makefiles des sous-répertoires et qui exécute
├── common            : répertoire des fichiers commun kernel / user
│   └── syscall.h     : API la fonction syscall et des codes de syscalls
├── kernel            : Répertoire des fichiers composant le kernel
│   ├── kinit.c       : fichier contenant la fonction de démarrage du noyau
│   ├── harch.h       : API du code dépendant de l'architecture
│   ├── harch.c       : code dépendant de l'architecture du SoC
│   ├── hcpu.h        : prototype de la fonction clock()
│   ├── hcpu.S        : code dépendant du cpu matériel en assembleur
│   ├── hcpc.c        : code dépendant du cpu matériel en c
│   ├── klibc.h       : API de la klibc
│   ├── klibc.c       : fonctions standards utilisées par les modules du noyau
│   ├── kpanic.h      : déclaration du tableau de dump des registres en cas d'exception
│   ├── kpanic.c      : fonction d'affichage des registres avant l'arrêt du programme
│   ├── ksyscalls.c   : Vecteurs des syscalls
│   ├── kernel.ld     : ldscript décrivant l'espace d'adressage pour l'édition de liens du kernel
│   └── Makefile      : description des actions possibles sur le code kernel : compilation et nettoyage
└── user             : Répertoire des fichiers composant l'application user
    ├── crt0.c        : fonctions d'interface entre kernel et user, pour le moment : _start()
    ├── main.c        : fonction principale de l'application
    ├── user.ld       : ldscript décrivant l'espace d'adressage pour l'édition de liens du user
    └── Makefile      : description des actions possibles sur le code user : compilation et nettoyage
```

Questions

- Dans quel fichier se trouve la définition des numéros de services tels que `SYSCALL_EXIT` ?
(Ces numéros sont communs au noyau et à l'application, alors ils ont été mis dans un répertoire commun aux deux exécutable.)
- Dans quel fichier se trouve le vecteur de syscall, c'est-à-dire le tableau `syscall_vector[]` contenant les pointeurs sur les fonctions qui réalisent les services correspondants aux syscall ? C'est un tableau indexé par les numéros de syscall qui permet de connaître la fonction à exécuter pour chaque numéro de service.
- Dans quel fichier se trouve le gestionnaire de syscalls ? (c'est de l'assembleur) et pourquoi est-ce de l'assembleur ?

Exercice

- Vous allez ajouter un appel système nommé `SYSCALL_CPUID` qui rend le numéro du processeur. Nous allons lui attribuer le numéro 4 (notez que ces numéros de services n'ont rien à voir avec les numéros utilisés pour le simulateur MARS). Pour ajouter un appel système, vous devez modifier les fichiers :
 - `kernel/hcpu.S` : pour ajouter `cpuid(void)` déjà écrite à l'étape précédente;
 - `kernel/hcpu.h` : pour le prototype de la fonction `cpuid()` (afin que gcc la connaisse pour la compilation de `kernel/ksyscall.c`);
 - `common/syscalls.h` : pour ajouter le numéro de syscall `SYSCALL_CPUID`;
 - `kernel/ksyscall.c` : pour ajouter la fonction `cpuid()` dans le vecteur de syscall.
- Modifiez aussi la fonction `main()` pour utiliser votre nouvel appel système. Vous pouvez utiliser le mode debug pour voir dans la trace d'exécution l'appel à

4. Ajout de la librairie C pour l'utilisateur

Objectifs de l'étape

L'application utilisateur n'est pas censée utiliser directement les appels système. Elle utilise une librairie de fonctions standards (la `libc` POSIX, mais pas uniquement) et ce sont ces fonctions qui réalisent les appels système. Toutes les fonctions de la `libc` n'utilisent pas les appels système. Par exemple, les fonctions `int rand(void)` ou `int strlen(char *)` (rendent, respectivement, un nombre pseudo aléatoire et la longueur d'une chaîne de caractères) n'ont pas besoin du noyau. Les librairies font partie du système d'exploitation mais elles ne sont pas dans le noyau.

Le terme « librairie » vient de l'anglais « library » qui signifie bibliothèque. On utilise souvent le mot librairie même si le sens en français n'est pas le même que celui en anglais. Disons que, dans notre contexte, les deux mots sont synonymes.

Normalement, les librairies système sont des « vraies » librairies au sens `gcc` du terme. C'est-à-dire des archives de fichiers objet (`.o`). Ici, nous allons simplifier et ne pas créer une vraie librairie, mais seulement un fichier objet `libc.o` contenant

toutes les fonctions. Ce fichier objet doit être lié avec le code de l'application.

L'exécutable de l'application utilisateur est donc composé de deux parties : d'un côté, le code de l'application et, de l'autre, le code de la librairie `libc` (+ `crt0`). Nous allons répartir le code dans deux répertoires:

- `uapp` pour les fichiers de l'application utilisateur
- `ulib` pour les fichiers qui ne sont pas l'application, c'est-à-dire la `libc`, le fichier `crt0.c` mais aussi le fichier `ldscript user.ld`.

Fichiers

```
4_libc/
├── Makefile      : Makefile racine qui invoque les Makefiles des sous-répertoires et qui exécute
├── common       : répertoire des fichiers commun kernel / user
│   └── syscalls.h : API la fonction syscall et des codes de syscalls
├── kernel       : Répertoire des fichiers composant le kernel
│   ├── kinit.c   : fichier contenant la fonction de démarrage du noyau
│   ├── harch.h   : API du code dépendant de l'architecture
│   ├── harch.c   : code dépendant de l'architecture du SoC
│   ├── hcpu.h    : prototype de la fonction clock()
│   ├── hcpua.S   : code dépendant du cpu matériel en assembleur
│   ├── hcpuc.c   : code dépendant du cpu matériel en c
│   ├── klibc.h   : API de la klibc
│   ├── klibc.c   : fonctions standards utilisées par les modules du noyau
│   ├── ksyscalls.c : Vecteurs des syscalls
│   ├── kernel.ld : ldscript décrivant l'espace d'adressage pour l'édition de liens du kernel
│   └── Makefile  : description des actions possibles sur le code kernel : compilation et nettoyage
├── uapp         : Répertoire des fichiers de l'application user seule
│   ├── main.c    : fonction principale de l'application
│   └── Makefile  : description des actions possibles sur le code user : compilation et nettoyage
└── ulib         : Répertoire des fichiers des bibliothèques système liés avec l'application user
    ├── crt0.c    : fonctions d'interface entre kernel et user, pour le moment : _start()
    ├── libc.h    : API pseudo-POSIX de la bibliothèque C
    ├── libc.c    : code source de la libc
    ├── user.ld   : ldscript décrivant l'espace d'adressage pour l'édition de liens du user
    └── Makefile  : description des actions possibles sur le code user : compilation et nettoyage
```

Questions

1. Pour ce petit système, dans quel fichier sont placés tous les prototypes des fonctions de la `libc`? Est-ce ainsi pour POSIX sur LINUX?

Exercice

- Vous allez juste ajouter la fonction `int cpuid()` dans la librairie `libc`.
- Au premier TP, vous deviez créer un petit jeu 'guess', vous pouvez en faire une application utilisateur, en utilisant cette fois les fonctions de la `libc`.

Gestionnaire d'interruptions

Rappel de cours

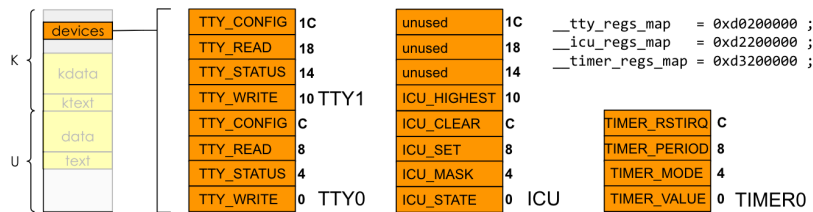
Il est fortement recommandé de lire les transparents, toutefois, nous avons mis ci-après quelques rappels utiles pour répondre aux questions du TD.

Rappel de cours

Contrôleurs de périphériques
IRQ et ISR
Routage et masquage des IRQ
Conséquence dans le noyau
Rappel sur les 3 registres du coprocesseur 0 impliqués
Questions

Contrôleurs de périphériques

- Dans cette séance, nous allons manipuler 3 contrôleurs de périphériques: le TTY que vous connaissez déjà et deux autres, l'ICU et le TIMER.
- Ces trois contrôleurs s'utilisent grâce à des registres mappés (c.-à-d. placés) dans l'espace d'adressage du MIPS.
 - Les registres du TTY sont placés à partir de l'adresse `0xd0200000`,
 - ceux de l'ICU à partir de l'adresse `0xd2200000`
 - et enfin ceux du TIMER à partir de l'adresse `0xd3200000`.
- Le rôle de ces registres est rappelé en partie dans ce texte et pour plus de détails, vous devez revoir le cours.
- Le choix des adresses de ces contrôleurs est fait par le créateur du matériel, elles ne peuvent pas être changées par le logiciel.
- Ces adresses sont données dans le fichier `ldscript` du kernel (`kernel.ld`) parce qu'elles ne sont utilisables que si le MIPS est en mode kernel (adresses > `0x80000000`).

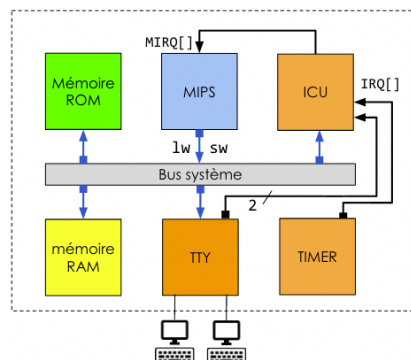


IRQ et ISR

- Les IRQ (Interrupt ReQuest)s sont des signaux électriques à 2 états (ON/OFF, on dit aussi Actif/Inactif ou encore Levé/Baissé).
- Les IRQ sont levés par les contrôleurs de périphériques pour prévenir d'un événement (fin de commande, arrivée d'une donnée, etc.).
- Quand une IRQ se lève, la conséquence est que le programme en cours d'exécution sur le processeur recevant l'IRQ est interrompu et qu'il est dérivé vers le noyau pour que ce dernier exécute l'ISR prévue pour l'IRQ.
- Les IRQ provoquent donc l'exécution d'ISR (Interrupt Service Routine) par le noyau.
- Les IRQ sont des signaux d'état qui doivent rester levés/activés tant qu'ils n'ont pas été acquittés par une ISR.
- Les ISR sont des fonctions qui reçoivent en argument un identifiant du contrôleur de périphérique qui a levé l'IRQ.
- Notez que ce n'est pas le processeur qui est interrompu, c'est bien le programme, car le processeur est seulement dérivé vers le noyau, mais il continue à travailler.
- Une ISR doit faire deux choses,
 - accéder aux registres du contrôleur de périphérique concerné pour faire ce que le périphérique demande
 - acquitter l'IRQ, c'est-à-dire demander au contrôleur de périphérique de baisser/désactiver son IRQ (puisque celle-ci a été traitée).
- La demande d'acquiescement est spécifique à chaque contrôleur de périphérique.
 - Pour le TTY, il faut lire le registre `TTY_READ`.
 - Pour le TIMER, il faut écrire dans le registre `TIMER_RSTIRQ`.

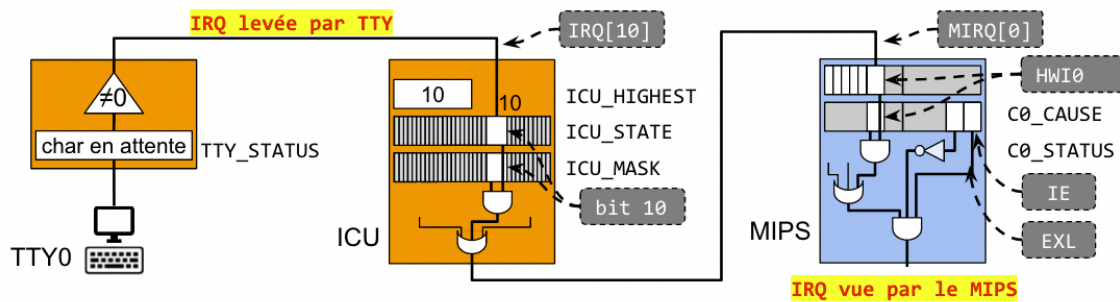
Routage et masquage des IRQ

- Sur le schéma de la plateforme des TP, on peut voir que seuls les composants TTY et TIMER peuvent lever des IRQ.
- Les IRQ de ces contrôleurs de périphériques sont envoyés au composant ICU qui va les combiner pour produire un unique signal IRQ pour le processeur.



- Une IRQ peut être masquée, c'est-à-dire que le processeur ne va pas interrompre le programme en cours.
- Le masquage peut être demandé à plusieurs endroits : dans le composant ICU et dans le processeur lui-même (parfois même dans le contrôleur de périphérique).

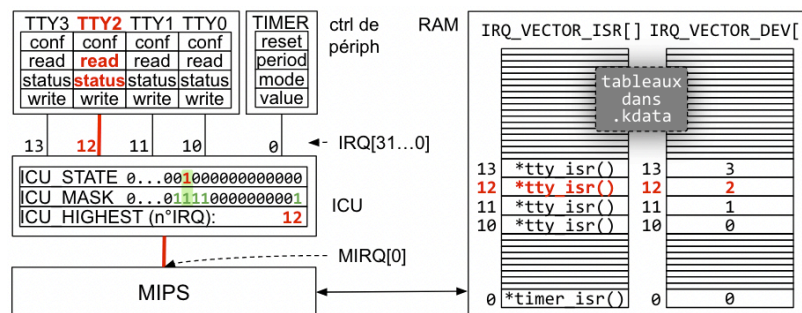
- Le masquage est demandé par le noyau, le plus souvent de manière temporaire, quand il doit exécuter un code critique qui ne doit surtout pas être interrompu.



- Sur le schéma ci-dessus, on voit que l'IRQ du TTY0 est reliée à l'entrée n° 10 de l'ICU, c'est un choix matériel qui n'est pas modifiable par logiciel. Son état est donc enregistré dans le bit n°10 du registre `ICU_STATE`.
- Il y a un `AND` avec le bit 10 du registre `ICU_MASK`. Si le bit 10 du registre `ICU_MASK` est à 0, alors la sortie du `AND` est 0 et l'IRQ est masquée (donc invisible pour le processeur).
- Le registre `ICU_HIGHEST` contient toujours le numéro de l'IRQ active la plus prioritaire, comme il n'y en a qu'une dans cet exemple, `ICU_HIGHEST` contient 10 (l'IRQ prioritaire, pour cette ICU, est l'IRQ active dont le numéro est le plus petit).
- La sortie IRQ de l'ICU est reliée à l'entrée 0 des 6 IRQs possibles du MIPS et sa valeur s'inscrit dans le registre `HWI0` du registre `c0_cause`.
- Il y a un `AND` avec le bit `HWI0` du registre `c0_status`. Si le bit `HWI0` du registre `c0_status` est à 0, alors la sortie du `AND` est 0 et l'IRQ est aussi masquée.
- Enfin, il y a un dernier `AND` avec le bit 0 de `c0_status` (correspondant au bit `IE` pour Interrupt Enable) qui permet de masquer globalement les IRQ et avec le `NOT` du bit 1 de `c0_status` (correspondant au bit `EXL` Exception Level).
- Quand le signal IRQ vue par le MIPS s'active (passe à 1), c'est que l'IRQ levée par le contrôleur de périphérique doit être prise en charge. Le programme en cours d'exécution est interrompu et dérivé vers `kentry` à l'adresse `0x80000180` et en même temps `C0_EPC ← PC+4`, `c0_cause.XCODE ← 0`, `c0_status.EXL ← 1` (notez que le nom officiel de `c0_status` est `C0_SR`, mais dans ce document, on utilise `c0_status` pour plus de clarté).

Conséquence dans le noyau

- Dans le schéma ci-après, à gauche c'est le matériel et à droite c'est un extrait de la RAM contenant les structures de données utilisées par le noyau pour la gestion des IRQ.

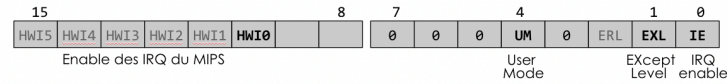


- À gauche, on voit que les IRQ venant des contrôleurs de périphériques sont connectés aux entrées d'IRQ de l'ICU.
 - Il y a 32 entrées possibles. Sur notre plateforme, par exemple l'IRQ du TTY2 est connectée à l'entrée 12 de l'ICU. Ce numéro d'entrée est le numéro qui identifie le contrôleur de périphérique.
 - Notez que le registre `ICU_MASK` est en lecture seule, c'est-à-dire qu'il ne peut pas être écrit directement. Pour modifier le contenu du registre `ICU_MASK`, il faut utiliser deux autres registres de l'ICU : `ICU_SET` et `ICU_CLEAR`. `ICU_SET` permet de mettre à 1 les bits de `ICU_MASK`, et `ICU_CLEAR` permet de les mettre à 0. Pour mettre à 1 le bit `i` du registre `ICU_MASK`, il faut écrire 1 dans le bit `i` du registre `ICU_SET`. Pour mettre à 0 le bit `j` du registre `ICU_MASK`, il faut aussi écrire 1 dans le bit `j` du registre `ICU_CLEAR`.
- À droite, il y a les deux tableaux que le noyau utilise pour connaître l'ISR à exécuter pour chaque numéro d'IRQ.
 - Ce couple de tableaux se nomme **vecteur d'interruption** et comme il y a 32 entrées d'IRQ dans l'ICU, ces tableaux ont 32 cases chacun.
 - Ici, le vecteur d'interruption est composé des tableaux `IRQ_VECTOR_ISR[]` et `IRQ_VECTOR_DEV[]`.
 - Le vecteur d'interruption est indexé par les numéros d'IRQ. Il contient deux informations:
 - dans la case n° `i` du tableau `IRQ_VECTOR_ISR[]`, on trouve le pointeur sur la fonction ISR à appeler si l'IRQ n° `i` est levée
 - dans la case n° `i` du tableau `IRQ_VECTOR_DEV[]`, on trouve le numéro de l'instance du périphérique.
 - Cette dernière information est nécessaire dans le cas des contrôleurs de périphérique multi-instances comme le TTY afin de savoir quel jeu de registres la fonction ISR doit utiliser.
 - En effet, il y a une fonction ISR unique à exécuter quel que soit le numéro du TTY. Dans cette plateforme, comme il y a 4 TTY, l'adresse de la fonction ISR est placée dans les cases 10, 11, 12, et 13 du tableau `IRQ_VECTOR_ISR[]`. Dans les cases 10, 11, 12, et 13 du tableau `IRQ_VECTOR_DEV[]`, on a les valeurs 0, 1, 2 et 3 qui correspondent bien au numéro d'instance des TTYs.

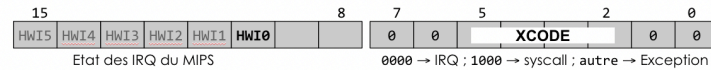
Rappel sur les 3 registres du coprocesseur 0 impliqués

- Nous rappelons ci-après les 3 registres du coprocesseur 0 utilisés au moment de l'entrée dans le noyau, quelle que soit la cause : syscall (vu la semaine dernière), interruption (TD de cette semaine) et exception (dans le cas de problèmes lors de l'exécution du programme comme la division par 0).
- On rappelle aussi que les seules instructions qui peuvent manipuler ces registres sont `mtc0` et `mfc0` pour, respectivement, les écrire et les lire.

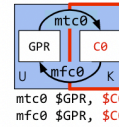
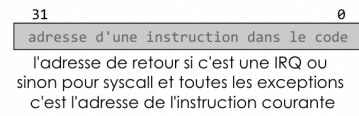
Le registre `c0_sr` (\$12) contient le mode d'exécution du MIPS et les autorisations d'IRQ



Le registre `c0_cause` (\$13) contient la cause d'entrée dans le noyau (si IRQ, syscall ou except)



Le registre `c0_epc` (\$14)



- Les bits `HWI0` des registres `c0_status` (aussi nommé `c0_sr`) et `c0_cause` contiennent respectivement le mask et l'état de l'entrée n° 0 d'interruption du MIPS.
- Les bits `UM`, `IE` et `EXL` sont liés au mode d'exécution du MIPS:
 - `UM` est le bit de mode du MIPS (1 = User Mode, 0 = Kernel Mode),
 - `IE` est le bit de masque général des interruptions (1 = autorisées, 0 = masquées)
 - et enfin `EXL` est le bit que le MIPS met à 1 à l'entrée dans le noyau pour informer d'un niveau exceptionnel et dans ce cas les bits `UM` et `IE` ne sont plus significatifs, si `EXL` est à 1 alors le MIPS est en mode kernel, et les interruptions sont masquées.

Questions

La majorité des réponses aux questions ci-après sont dans le rappel du cours donné au début de cette page, c'est voulu.

- À quelles adresses dans l'espace d'adressage sont placés les registres des 3 contrôleurs de périphériques de la plateforme et comment le kernel les connaît ?
- Que signifie l'acronyme I.R.Q. ?
- Une IRQ est un signal électrique, combien peut-il avoir d'états ?
- Qu'est-ce qui provoque une IRQ ?
- Les IRQ relient des composants sources et des composants destinataires, quels sont ces composants ? Donnez un exemple.
- Que signifie masquer une IRQ ?
- Quels composants peuvent masquer une IRQ ?
- Est-ce qu'une application utilisateur peut demander le masquage d'une IRQ ?
- Que signifie l'acronyme I.S.R. ?
- Dans la plateforme des TP, sur quelles entrées de l'ICU sont branchées les IRQ venant des TTYs et du TIMER ?
- Quelle valeur faut-il avoir dans le registre `ICU_MASK` si on veut recevoir seulement les IRQ venant des 4 TTYs, dans le cas de la plateforme utilisée en TP ? Donnez le nombre en binaire et en hexadécimal.
- L'écriture dans `ICU_MASK` n'est pas possible, comment modifier ce registre pour mettre à 1 le bit 10 ?
- Sur une plateforme (autre que celle des TP) sur laquelle on aurait un TTY0 sur l'entrée 5, un TIMER sur l'entrée 2, et un autre TTY1 sur l'entrée 14. Que doit-on faire pour que seuls le TTY1 et le TIMER soient démasqués et que TTY0 soit masqué ?
- Si on définit dans le code C:

```
struct icu_s {
    int state;           // état des IRQ à l'entrée de l'ICU
    int mask;            // masque des IRQ
    int set;             // registre de mise à 1 des bits du registre mask
    int clear;           // registre de mise à 0 des bits du registre mask
    int highest;         // numéro de l'IRQ active la plus prioritaire
    int unused[3];       // 3 registres non utilisés
};
extern volatile struct icu_s __icu_regs_map; // déclaration de cette struct pour gcc
```

Ecrivez la fonction mettant à 1 le bit n° `irq` du registre `ICU_MASK`: `void icu_set_mask (int irq)`

- Si les 3 IRQ de la question précédente se lèvent au même cycle, quelles seront les valeurs des registres `ICU_STATE`, `ICU_MASK` et `ICU_HIGHEST` ?
- Dans quel mode est le processeur quand il traite une IRQ ?
- Que fait le processeur lorsqu'il reçoit une IRQ masquée ?
- Que signifie acquitter une IRQ ?
- Qui demande l'acquiescement à qui ?
- Comment demande-t-on l'acquiescement ? Donnez au moins un exemple.
- Est-ce qu'une IRQ peut se désactiver sans intervention du processeur ?
- Est-ce qu'une IRQ peut ne pas être attendue par le noyau ?
- Quelle est la valeur du champ `XCODE` du registre `c0_cause` à l'entrée dans le noyau en cas d'interruption ?
- Quelle est la valeur écrite dans le registre `c0_epc` à l'entrée dans le noyau en cas d'interruption ?
- Que se passe-t-il dans le registre `c0_status` à l'entrée dans le noyau en cas d'interruption et quelle est la conséquence ?
- La routine `kentry` (entrée du kernel à l'adresse `0x80000180`) appelle le gestionnaire d'interruption quand le MIPS reçoit une IRQ non masquée, que fait ce gestionnaire d'interruption ?
- À l'entrée dans le noyau, `kentry` analyse le champ `XCODE` du registre de `c0_cause` et si c'est 0 alors il saute au code donné ci-après (ce n'est pas exactement le code que vous pouvez voir dans les fichiers sources pour que ce soit plus facile à comprendre).

```
irq_handler:
    addiu    $29,    $29,    -23*4      // 23 registers to save (18 tmp regs+HI+L0+$31+EPC+SR)
    mfc0     $27,    $14                // $27 ← EPC (addr of syscall instruction)
    mfc0     $26,    $12                // $26 ← SR (status register)
    sw       $31,    22*4($29)          // $31 because, it is lost by jal isrcall
```

```

sw      $27, 21*4($29)           // save EPC (return address of IRQ)
sw      $26, 20*4($29)           // save SR (status register)
mtc0    $0, $12                  // SR <- kernel-mode without INT (UM=0 ERL=0 EXL=0 IE=0)
sw      $1, 1*4($29)             // save all temporary registers including HI and LO
sw      $2, 2*4($29)
[etc. pour les autres sauvegardes des registres temporaires]

jal      isrcall                  // call the irq handler fonction écrite en C

lw      $1, 1*4($29)             // restore all temporary registers including HI and LO
lw      $2, 2*4($29)
[etc. pour les autres restaurations des registres temporaires]
lw      $26, 20*4($29)           // get old SR
lw      $27, 21*4($29)           // get return address of syscall
lw      $31, 22*4($29)           // restore $31
mtc0    $26, $12                 // restore SR
mtc0    $27, $14                 // restore EPC
addiu   $29, $29, 23*4           // restore the stack pointer
eret                                // jr C0_EPC AND C0_SR.EXL <= 0

```

Pourquoi, ne pas sauver les registres persistants ?

28. La fonction `isrcall()` a pour mission d'appeler la bonne ISR. Dans le code qui suit (extrait du fichier `kernel/harch.c`), on voit d'abord la déclaration de la structure qui décrit les registres présents dans l'ICU. En fait c'est un tableau de structures parce qu'il y a autant d'instances d'ICU que de processeurs (donné par `NCPUS`), ici, il y a un seul processeur MIPS, donc `NCPUS=1`.

```

struct icu_s {
    int state;           // state of all IRQ signals
    int mask;            // IRQ mask to chose what we need for this ICU
    int set;             // IRQ set --> enable specific IRQs for this ICU
    int clear;          // IRQ clear --> disable specific IRQs for this ICU
    int highest;        // highest priority IRQ number for this ICU
    int unused[3];      // these 3 registers are not used
};
extern volatile struct icu_s __icu_regs_map[NCPUS];

static int icu_get_highest (int icu) {
    return __icu_regs_map[icu].highest;
}

static void icu_set_mask (int icu, int irq) {
    __icu_regs_map[icu].set = 1 << irq;
}

void isrcall (void) {
    int irq = icu_get_highest (cpuid());
    irq_vector_isr[irq] (irq_vector_dev[irq]);
}

```

La déclaration `extern volatile struct icu_s __icu_regs_map[NCPUS];` informe le compilateur que le symbole `__icu_regs_map` est défini ailleurs et que c'est un tableau de structures de type `struct icu_s`. Ainsi, le compilateur `gcc` sait comment utiliser la variable `__icu_regs_map`.

Dans quel fichier est défini `__icu_regs_map` ? Que font les fonctions `icu_get_highest()`, `icu_set_mask()` et `isrcall()` ? Comment s'appelle le couple de tableaux `irq_vector_isr[irq]` et `irq_vector_dev[irq]` et combien de cases ont-ils ?

29. Si `ICU_HIGHEST` contient 10 (dans le cas de notre plateforme) que doit faire la fonction `isrcall()` ?
 30. Que fait la fonction `icu_set_mask (int icu, int irq)` ?
 31. Les registres du TIMER sont définis dans le code du noyau de la façon suivante :

```

struct timer_s {
    int value;           // timer's counter : +1 each cycle, can be written
    int mode;            // timer's mode : bit 0 = ON/OFF ; bit 1 = IRQ enable
    int period;          // timer's period between two IRQ
    int resetirq;        // address to acknowledge the timer's IRQ
};
extern volatile struct timer_s __timer_regs_map[NCPUS];

```

Écrivez le code de la fonction `static void timer_init (int timer, int tick)` qui initialise la période du timer n° `timer` avec l'entier nommé `tick` et active les IRQ si la période donnée est non nulle.

32. La configuration des périphériques et des interruptions est faite dans la fonction `arch_init()` appelée par `kinit()`. Écrivez les instructions C permettant d'ajouter le TIMER dans le noyau avec un tick de 1000000 (1 million de cycles). Il faut (1) initialiser le timer ; (2) démasquer l'IRQ venant du timer dans l'ICU, elle connectée sur son entrée n°0 ; (3) initialiser le vecteur d'interruption avec la fonction `timer_isr` pour ce timer 0.

La suite en TP...

Gestionnaire d'interruptions

IMPORTANT

Avant de faire cette séance, vous devez avoir lu les documents suivants :

- Cours sur le gestionnaire d'interruption et les threads : obligatoire
- Document sur l'assembleur du MIPS et la convention d'appel des fonctions : recommandé, mais déjà lu
- Documentation sur le mode kernel du MIPS32 : fortement recommandé

1. Game over simple
2. Game over avec décompteur
3. Évaluation de la durée d'une ISR

1. Game over simple

Dans le TP9, vous avez réalisé un petit jeu dans lequel vous deviez deviner un nombre tiré au hasard. Ce jeu avait été mis dans `kinit` parce qu'à ce moment, il n'y avait pas encore d'application utilisateur. Nous vous proposons de mettre le jeu dans l'application user et de limiter le temps pendant lequel vous pouvez jouer. Nous allons vous guider pas-à-pas.

Récupérez l'archive du code du tp3, placez-la dans le répertoire `k06` et décompressez-la. Les commandes ci-dessous supposent que vous avez mis l'archive dans le répertoire `k06`

```
cd ~/k06
tar xvzf tp3.tgz
cd tp3/1_gameover
```

Le code de l'application est le suivant (dans `uapp/main.c`)

```
#include <libc.h>
int main (void)
{
    int guess;
    int random;
    char buf[8];
    char name[16];

    fprintf(0, "Tapez votre nom : ");
    fgets(name, sizeof(name), 0);
    if (name[strlen(name)] == '\n')
        name[strlen(name)] = 0;
    srand(clock()); // start the random generator with a "random" seed.

    random = 1 + rand() % 99;
    fprintf(0, "Donnez un nombre entre 1 et 99: ");
    do {
        fgets(buf, sizeof(buf), 0);
        guess = atoi (buf);
        if (guess < random)
            fprintf(0, "%d est trop petit: ", guess);
        else if (guess > random)
            fprintf(0, "%d est trop grand: ", guess);
    } while (random != guess);

    fprintf(0, "\nGagné %s\n", name);
    return 0;
}
```

1. Essayez le jeu (dans le répertoire `tp3/1_gameover`) : tapez `make exec` comme vous pouvez le constater, vous avez le temps de jouer.
2. Dans la version précédente du gestionnaire de syscall, nous avions masqué les IRQ en écrivant `0` dans le registre `c0_status` (registre \$12 du coprocesseur 0). Cela avait pour conséquence de mettre tout à 0, entre autres le bit `IE`. Il faut modifier ça, parce que sinon, lorsque l'utilisateur demandera à lire le clavier avec l'appel système `fgets()`, l'IRQ venant du timer ne sera jamais prise en compte (`TOD01`), ensuite au retour de la fonction qui réalise l'appel système, il faut masquer les IRQ pour ne pas avoir d'interruption pendant la restauration des registres jusqu'au `eret` qui fait sortir du kernel.

```
syscall_handler:
    addiu    $29,    $29,    -8*4           // context for $31 + EPC + SR + syscall_code + 4 args
    mfc0     $27,    $14                   // $27 <- EPC (addr of syscall instruction)
    mfc0     $26,    $12                   // $26 <- SR (status register)
    addiu    $27,    $27,    4             // $27 <- EPC+4 (return address)
    sw       $31,    7*4($29)              // save $31 because it will be erased
    sw       $27,    6*4($29)              // save EPC+4 (return address of syscall)
    sw       $26,    5*4($29)              // save SR (status register)
    sw       $2,     4*4($29)              // save syscall code (useful for debug message)
    // TOD01: remplacez "mtc0 $0, $12" par 2 autres pour mettre 1 dans les bits c0_sr.HWI0 et
    c0_sr.IE
    // vous pouvez utiliser $26
    mtc0     $0,     $12                   // SR <- kernel-mode without INT (UM=0 ERL=0 EXL=0 IE=0)

    la       $26,    syscall_vector       // $26 <- table of syscall functions
    andi     $2,     $2,    SYSCALL_NR-1  // apply syscall mask
    sll      $2,     $2,    2              // compute syscall index (multiply by 4)
    addu     $2,     $2,    $2             // $2 <- & syscall_vector[$2]
```

```

    lw      $2,      ($2)          // at the end: $2 <- syscall_vector[$2]
    jalr    $2              // call syscall function

// TODO2: Il faut mettre 0 dans SR pour masquer les interruptions
lw      $26,      5*4($29)        // get old SR
lw      $27,      6*4($29)        // get return address of syscall
lw      $31,      7*4($29)        // restore $31 (return address of syscall function)
mtc0     $26,      $12            // restore SR
mtc0     $27,      $14            // restore EPC
addiu    $29,      $29,      8*4   // restore stack pointer
eret                                // return : jr EPC with EXL <- 0

```

```

// TODO1: remplacez "mtc0 $0, $12" par 2 autres pour mettre 1 dans les bits c0_sr.HWI0 et
c0_sr.IE
    li      $26,      0x401        // next value of SR
    mtc0    $26,      $12          // SR <- kernel-mode with INT (HWI0=1 UM=0 ERL=0 EXL=0
IE=1)

// TODO2: Il faut mettre 0 dans SR pour masquer les interruptions
mtc0     $0,      $12            // SR <- kernel-mode without INT (UM=0 ERL=0 EXL=0
IE=0)

```

3. Ouvrez le fichier `kernel/kinit.c`. Dans cette fonction, on appelle `archi_init()` avec en paramètre un nombre qui va servir de période d'horloge. Le simulateur de la plateforme sur les machines de la PPTI va environ à 3.5MHz. Combien de secondes demande-t-on dans ce code ?

`arch_init (30*3500000);` about 30 secondes with this simulator (3.5MHz)

4. Ouvrez le fichier `kernel/harch.c` et vous allez devoir remplir 3 fonctions pour configurer le timer: `arch_init()`, `timer_init()` et `timer_isr()` (pour trouver ces fonctions cherchez le mot `TODO`)

```

void arch_init (int tick)
{
    // TODO A remplir avec 4 lignes :
    // 1) appel de la fonction timer_init pour le timer 0 avec tick comme période
    // 2) mise à 1 du bit 0 du registre ICU_MASK en utilisant la fonction icu_set_mask()
    // 3) initialisation de la table irq_vector_isr[] vecteur d'interruption avec timer_isr()
    // 4) initialisation de la table irq_vector_dev[] vecteur d'interruption avec 0
}

static void timer_init (int timer, int tick)
{
    // TODO A remplir avec 2 lignes :
    // 1) initialiser le registre period du timer n°timer avec la période tick (reçus en argument)
    // 2) initialiser le registre mode du timer n°timer avec 3 (démarré le timer avec IRQ
    // demandée) si la période est non nulle
}

static void timer_isr (int timer)
{
    // TODO A remplir avec 3 lignes :
    // 1) Acquiescer l'interruption du timer en écrivant n'importe quoi dans le registre resetirq
    // 2) afficher un message "Game Over" avec kprintf()
    // 3) appeler la fonction kernel exit() (c'est une sortie définitive ici)
}

```

```

void arch_init (int tick)
{
    timer_init (0, tick);          // sets period of timer n'0 (thus for CPU n'0) and starts
it
    icu_set_mask (0, 0);          // [CPU n'0].IRQ <-- ICU.PIN[0] <- Interrupt signal timer
n'0
    irq_vector_isr [0] = timer_isr; // tell the kernel which isr to exec for ICU.PIN n'0
    irq_vector_dev [0] = 0;        // device instance attached to ICU.PIN n'0
}

static void timer_init (int timer, int tick)
{
    __timer_regs_map[timer].period = tick;    // next period
    __timer_regs_map[timer].mode = (tick)?3:0; // timer ON with IRQ only if (tick != 0)
}

static void timer_isr (int timer)
{
    __timer_regs_map[timer].resetirq = 1;      // IRQ acknowledgement to lower the interrupt
signal
    kprintf ("\nGame Over\n");
    exit(1);
}

```

2. Game over avec décompteur

Dans ce qui précède, l'exécution de l'ISR du Timer est fatale puisqu'elle stoppe l'application après l'affichage de "Game Over!". Nous vous proposons de modifier l'ISR afin d'avoir un comportement un peu plus réaliste. Dans cette nouvelle version, l'ISR du timer décrémente un compteur alloué dans une variable globale du noyau, puis elle revient dans l'application tant que ce compteur est différent de 0. Donc, dans l'ISR du timer si le compteur est différent de 0, elle affiche un message avec la valeur du compteur, sinon elle affiche "game over!" et stoppe l'application, comme dans l'exercice précédent.

Par exemple, au lieu d'afficher:

```
|_|_/_v'\_/_/_/
|_/_/(\_/_/)\_/_/
|_/_/\_/_/x\_/_/x\_/_/
```

```
Tapez votre nom : Moi
Donnez un nombre entre 1 et 99: 45
45 est trop grand: 20
20 est trop grand:
0 est trop petit:
Game Over
[105002991] EXIT status = 1
```

l'application pourrait afficher:

```
|_|_/_v'\_/_/_/
|_/_/(\_/_/)\_/_/
|_/_/\_/_/x\_/_/x\_/_/
```

```
Tapez votre nom : Moi
Donnez un nombre entre 1 et 99: 45
45 est trop grand: 20
20 est trop grand:
..3 : 12
12 est trop petit: 15
15 est trop petit:
..2 :
..1 :
Game Over
[115002778] EXIT status = 1
```

kernel/harch.c

```
extern void arch_init (int tick, unsigned quantum);
```

kernel/harch.c

```
static unsigned timer_quantum;
static void timer_init (int timer, int tick, unsigned quantum)
{
    __timer_regs_map[timer].resetirq = 0;    // to delete previous untratted IRQ
    __timer_regs_map[timer].period = tick;    // next period
    __timer_regs_map[timer].mode = (tick)?3:0; // timer ON with IRQ only if (tick != 0)
    timer_quantum = quantum % 100;           // %100 to avoid aberrant value
}
static void timer_isr (int timer)
{
    __timer_regs_map[timer].resetirq = 1;
    if (timer_quantum) {
        kprintf ("\n...%d : ", timer_quantum--);
    } else {
        kprintf ("\nGame Over\n");
        exit(1);
    }
}
```

kernel/kinit.c

```
void kinit (void)
{
    [...]
    arch_init (3*3500000, 10); // tick is about 3 seconde, quantum is then about 30 secondes
    [...]
}
```

3. Évaluation de la durée d'une ISR

Dans cet usage du TIMER, les ISR ne sont pas fatales, sauf la dernière. En utilisant le mode debug (make debug) et le fichier `trace0.S`, déterminez la durée en cycles du traitement par le noyau d'une IRQ du timer. Ce n'est pas exactement la même durée pour toutes les IRQ.

Pour cette question, il faut commenter les affichages dans l'ISR, changer la valeur du tick pour voir plus d'IRQ (par exemple 1000) et exécuter en mode debug puis regarder la trace dans `trace0.S`, il faut chercher un `kentry` correspondant à une IRQ et chercher l'instruction `eret` qui marque la fin du traitement.

- La durée mesurée est de l'ordre de 430 cycles pour la première (vers le cycle 3600).